FUNCTIONAL + TECHNICAL DECISION REPORT

Protosim + Protoduino

An AI toolbox grounded in executable evidence: persistent GDB debugging, model-independent MCP tools, source-aware inspection, configurable AVR profiles, environment diagnosis, reproducible development and migration.

Version 2.1 · 12 November 2026 · reviewed edition

Prepared for Johannes Klarenbeek and the development team

16 of 25

findings of edition 2.0 re-run here; all 16 hold, six of them worse than stated

GDB works

avr-gdb 15 against `protosim -d`: break, step, inspect, watch, interrupt

3

statements of edition 2.0 corrected, none of them a design decision

32

findings added, most from building a bootloader and module loader on Protoduino

What this edition is

- Edition 2.0 was written from source reading and seven host checks; no AVR compiler, simulator run or avr-gdb was available to it. This edition re-checks it with avr-gcc 7.3.0, Protosim built at the pinned revision, its pinned simavr, avr-gdb 15.0.50 and the MCP server driven over stdio.
- The design of edition 2.0 stands. Its decisions, requirements and gates are kept; where a statement was wrong or a caveat was missing, the text is changed in place and marked 2.1 .
- It adds what a parallel piece of work found: a streaming ELF bootloader and run-time module loader for Protoduino, built and run in simulation (`protoduino/docs/elf-loader-design.pdf`). That work exercised Protosim, simavr and Protoduino harder than any example does.
- Still a design specification, not a release certification. Nothing ran on hardware; PlatformIO was not available, so `compile_firmware` was read, not run. No repository source file was changed.

EXECUTED run for this edition on the simulated AVR, with avr-gdb or through MCP ISOLATED host check of edition 2.0 SOURCE code path read at the pinned revision PROPOSED requirement or recommendation 2.1 new or changed in this edition

Contents

00 Review of edition 2.0

01 Summary, scope, capabilities, requirements, workflows, architecture (to 06)

07 GDB, sessions, MCP contract, inspection (to 10)

11 Doctor (to 12)

13 Profiles, memory, migration, terminal UI (to 16)

17 Native modules

18 Verification, hardware, roadmap, decisions (to 21)

22 Findings (to 28)

29 Checks executed

30 Sources, references, examples (to 32)

00 Review of edition 2.0 2.1

Edition 2.0 proposes a host-side controller that gives a coding agent a persistent, source-aware debugger for a simulated AVR, and it lists 25 defects it found on the way by reading. The question for this review was whether that holds up when the code is run.

00.1 Verdict

- **The architecture is right, and its foundation works.** avr-gdb 15.0.50 connected to `protosim -d` at the pinned revision and did everything the design needs: a source breakpoint in `main`, stepping by line and by instruction, a RAM variable and a flash constant evaluated by name, a stack frame with file and line, a data watchpoint that reported old and new value, and an interrupt of a running target. It did the same on the ATmega2560. Edition 2.0 could only assume this.
- **The findings are accurate.** Of the 25, 16 were re-run here and every one behaved as described; six more were re-read against the source and stand (PS03, PS06, PD03, PD04, PD07, PD10); three were not re-checked (PS04, PD05, PD06) and nothing contradicts them. Six turned out worse than stated (00.3).
- **The most important omission is how a run ends.** On Linux, and by the same code path on macOS, a run that stops on `--max-steps` with the default UART bridge is killed by SIGINT from inside Protosim. The existing MCP tool makes `max_steps` mandatory, so its normal result is the error path: unparsed text, no failure flag. An agent using the tool as documented never sees a structured result. This belongs at the top of gate G0 (PS11, MC05).
- **The runtime-module part is overtaken.** Edition 2.0 treats a loader as future work with a staged plan. A bootloader and module loader now exist as running code in simulation. They implement stage L1 of that plan, without a package format, and add things the plan did not ask for: cooperative loading, and a module refused before the first flash write (17).

00.2 Corrections to edition 2.0

#	Edition 2.0 said	What is the case	Where
C1	PS01: a CPU crash “can exit successfully”, exit status 0 in both branches.	True for a headless run (`--uart0-out`). With the default UART bridge the same run, like every other ending, is killed by SIGINT: status 130 in a shell, a signal and no code in Node. “Map faults to nonzero exits” is not enough; the shutdown path itself has to be fixed first. EXECUTED	22, PS01, PS11
C2	09 and 32: connect the “existing locally built MCP server” at `mcp-server/build/index.js` ; the current tools stay available.	The path is right for a `tsc` build. But the repository also carries two committed builds, `mcp-server/dist/index.js` (a 716 KB bundle) and `mcp-server/src/index.js` , and the package installs the bundle as `/usr/lib/protosim/mcp/index.js` . Which server a user runs depends on how Protosim was installed, and the committed ones can lag behind `index.ts` . EXECUTED SOURCE	03, 32, MC06
C3	07: use `-exec-interrupt` “when supported in the selected MI mode”.	Made concrete: in the default mode avr-gdb gave no answer to `-exec-interrupt` while the target ran. After `-gdb-set mi-async on` , issued before connecting, it answered and the target stopped. The controller should set it always and keep SIGINT to gdb as the fallback, which also worked. EXECUTED	07

Nothing else in edition 2.0 was found to be wrong. Its arithmetic (65,024 bytes of usable SRAM in a 64 KiB data space), its MI command names, its description of the batch options and of the three existing MCP tools, and the pinned revisions all check out.

00.3 Findings that are worse than stated

ID	Edition 2.0	Observed
PS02	A paused GDB poll is counted as a step and instruments unexecuted code.	It also spends the step budget. With `--max-steps 700` and the CPU stopped at a breakpoint, Protosim announced “MAX-STEPS reached” and exited after 18.3 s in which no instruction ran. A debug session with a step limit ends itself while the developer reads.
PS07	The listener is not restricted to loopback; a failed start still prints “listening”.	Both confirmed: the debug port accepted a connection on the machine's non-loopback address, and a second instance on the same port printed “GDB server listening on port 1234” although its bind had failed.
PS09	The boot HEX parser accepts a record with a bad checksum.	It also accepts data after the end-of-file record, a line shorter than its length byte says (it reads on into the line end), and non-hex characters. All four return 0.
MC02	Two regular expressions do not match current output.	End to end through MCP: a run that hit its breakpoint returned `"breakpoints_hit":{}` and `"hot_path":null`. Only the coverage total was parsed.
PD01	`process_destroy()` leaves cleanup unscheduled.	It is not only `process_destroy()`. A process that simply ends is never finalized either: after 2.5 s its `PT_FINALLY` block had not run and its state was still EXITING.
PD09	A process struct above 255 bytes has no size assertion.	The compiler says “large integer implicitly truncated” and builds. A process struct of 316 bytes gets a descriptor that says 60.

00.4 What edition 2.0 did not have

- **How Protosim terminates** (PS11), that it has no `--help` or `--version` (PS12), and seven more defects around bootloaders, HEX loading, UART injection and reset (PS13 to PS19). Sections 22 and 23.
- **What simavr does differently from a chip** (AV02 to AV09). Edition 2.0 warns that writing simulator memory directly is not a loader test. It does not say that simavr also accepts SPM from anywhere, needs no erase, has no read-while-write state and programs a page in no time, so a loader that goes through the real self-programming path can still pass in simulation and fail on a chip. Section 25.
- **Scheduler and serial-path defects in Protoduino** beyond PD01 to PD04: polls starve events, `CC_EXTERN` defines variables, the serial buffers are 16 bytes on every part, the serial process strands output, and more (PD11 to PD22). Section 28.
- **A blocker for its own primary target.** `sys/clock.h` stops with `#error` for the ATmega2560, so nothing that uses a timer builds for the stock Mega profile the plan starts with (PD17).
- **The existing agent integration.** The repository ships `SKILL.md`, instructions for language models on using the CLI, and its tests are two PowerShell scripts. Both have to move with the tools (03).
- **Debug information in practice.** Source-aware inspection needs an ELF built for it. The Arduino tool chain builds with link-time optimisation by default and the pinned compiler is avr-gcc 7.3; what DWARF such a build carries has to be measured before decoders rely on it (10).

00.5 How this aligns with the ELF loader work

Edition 2.0 asks for	The loader work has	Difference to settle
L0: whole firmware through the normal load path	A bootloader that takes the unmodified `.elf` over XMODEM-CRC, refuses a wrong file before erasing, and commits with an image record. 13 scenarios pass in simavr.	Protosim cannot drive it yet: no binary input (PS15), a reset does not return to the loader (PS18), the STK500 thread always starts (PS19).

Edition 2.0 asks for	The loader work has	Difference to settle
L1: host-linked package for a reserved flash slot, with ABI metadata, validated and written by the device	Exactly this, with the ELF file as the package: a module is linked for a slot against one kernel build and starts with a descriptor. 9 scenarios pass, on Protoduino's unmodified scheduler, also on Optiboot.	Edition 2.0 lists initialised data and zero-fill; the loader forbids static data in a module instead, at link time. No whole-module checksum is stored after programming.
L2: bounded on-device relocation of `ET_REL` objects	Deliberately not done. A fix-up stream is proposed that needs no relocator on the device.	Decide whether L2 is still wanted.
L3: lifecycle, replacement, stale references	Start, module table, restart after reset, a second module beside a running one. Removal is half built.	PD04 (stale event pointers) and PD01 apply directly to unload.
Register module symbols with GDB after loading	Not done, and simple: a module is an `ET_EXEC` file at absolute addresses, so `add-symbol-file module.elf` needs no section arguments.	A controller task (10).

00.6 Ideas from this report to take into Protoduino

- **Host decoders for the scheduler** (F09, section 10). Finding why the loader's “module started” event disappeared took RAM dumps read by hand; a process-list and event-queue view at a stop would have shown a full queue at once.
- **Generation-checked process handles or purging events on teardown** (PD04). Module unload reuses the RAM block of a slot, which is the exact case PD04 describes.
- **A compile-time assertion on the size of a process struct** (PD09), extended to the RAM a module slot provides.
- **Honest outcomes** (F08). The loader already separates “loaded”, “nobody sent” and “refused, with a code”; the kernel side should report through the same vocabulary of stop reasons this report defines.
- **The evidence bundle** (F13) for the loader's 22 scenarios, in place of the ad hoc result files it has now.
- **A removal-safe poll sweep** (PD02) before modules can be unloaded while others are polled.

In the other direction, the loader work gives this toolbox three requirements it lacked: binary-safe file transfer to the simulated UART with a real protocol (F15), comparison of simulated flash against an independent reading of the ELF as a pass criterion (18), and a measurement of scheduler fairness, the turns a heartbeat process gets while something else runs, as a runtime metric (18).

01 Decision summary

The proposed toolbox lets a coding agent inspect the development environment, build firmware, operate a persistent simulated AVR, connect execution to source, and demonstrate that a change satisfies explicit requirements. Protosim remains an independently useful simulator and CLI. Protoduino remains the embedded runtime and application framework. A shared host-side controller adds GDB and MCP access without coupling either project to a particular language model.

Decision	Recommended direction
Preserve the foundations	Retain the CLI, batch runs, UART bridges, profiling and coverage. Correct misleading outcomes and instrumentation before relying on them for agent decisions. 2.1 Start with how a run terminates (PS11): until that is fixed no outcome can be trusted.
Use GDB as the foundation	Use avr-gdb through GDB/MI for source-level debugging; use simavr's remote stub for machine access. Add a narrow simulator channel for cycles, scheduled inputs and profiling. 2.1 The pairing was run and works (07).

Decision	Recommended direction
Make MCP the first integration	Extend the existing MCP server with persistent sessions. Claude Code, Codex and other compatible clients supply the AI and repository editing.
Keep inspection outside firmware	Read symbols, types and memory on the host. Protoduino-aware decoding does not require a firmware debug agent.
Make setup inspectable	Add `protosim --doctor`, a `doctor` alias, JSON output and an equivalent MCP tool. Report capabilities and failures with evidence.
Separate real and virtual targets	Start with stock devices, then qualify expanded memory profiles. Treat 1 MiB flash as an experimental engineering target, not an already validated maximum.
Treat the simulator as a model 2.1	Record, per result, which behaviour of a real chip simavr does not model (25). A flash-programming result from simulation is evidence of logic and addressing, not of flash discipline or timing.

What makes the combination distinctive. The strongest product opportunity is a complete evidence chain: environment, source revision, build, machine state, observable behaviour, resource budget, regression result. The same tools should support a human debugger, an AI coding agent, automated CI, and measured migration to smaller AVR targets. This is an intended differentiator; no market-wide uniqueness study or comparative benchmark was performed.

Recommended first demonstration

A coding agent diagnoses its tools, builds an unmodified Protoduino example, starts a persistent GDB session, reproduces a scheduler or terminal defect, inspects its cause, applies a source fix, and reruns a saved regression. 2.1 Two defects are now ready-made for it, each with a probe that fails today and a one-line fix: a process that ends is never finalized (PD01), and a process that polls itself stops all events (PD14). Both are reproduced by `protoduino/docs/elfloader/prototype/probes/run_probes.sh`. Expanded memory can follow without blocking this demonstration; run-time module loading no longer needs to wait for it, because it exists (17).

02 Scope, evidence and source revisions

The second edition incorporated the agreed GDB foundation, direct MCP use, provider independence and the environment doctor, and replaced the earlier Haiku-centred architecture with a reusable controller. All API names, CLI options and schemas described as proposed are requirements for future implementation, not claims that they exist today.

Reviewed repository	Pinned revision
Protosim	`6fadc6fcafd5bb14096436a3e1c87fef4a28cb05`
Protoduino standalone	`a3dbbf324703ebc1d26bd9d811deee56439964fc`
simavr used by Protosim	`a6fc4cc754ebf4ffb1e615ba7c2ab8a6d6f8b967`
Protoduino submodule in Protosim	`78d05ac0f5e8a448776e48b4bf3839b031ce13c0`; not audited, and not initialised in a fresh checkout
ELF loader work 2.1	Protoduino branch `ccr-8617b1c3-gjnqlx`, commit `8fdccda`: `docs/elf-loader-design.pdf`, `docs/elfloader/`

Evidence vocabulary. *Executed* 2.1: run for this edition, on the simulated AVR with the unmodified sources, with avr-gdb against `protosim -d`, or through the MCP server over stdio. *Isolated reproduction*: a small host test of edition 2.0 that executed the actual parser or extracted functions with controlled fixtures. *Source-*

confirmed: the relevant code path was read at the listed revision; this is not an end-to-end reproduction. *Risk / limitation*: a conditional failure or supported-design boundary. *Proposed*: a new feature, contract or engineering recommendation.

Tools of this edition. avr-gcc 7.3.0 with avr-libc, GNU gdb 15.0.50 built for AVR (Ubuntu package `gdb-avr`), Node 22, Protosim and simavr built from the pinned revisions on Linux. Not available: PlatformIO, the Arduino core, any hardware. No expanded-memory firmware and no model benchmark was exercised.

Units: KiB = 1,024 bytes; MiB = 1,048,576 bytes. The discussion's "2650" means ATmega2560. "ATTiny8" is an illustrative small target, not an exact device selection; the loader work takes the ATtiny25/45/85 family. Source keys [S1]–[S12] and [R1]–[R10] are listed in sections 30 and 31.

03 Current capabilities

Area	Observed implementation	Additional work
Simulator	ELF/HEX loading; `-m` MCU and `-f` frequency; simavr execution; `-d` starts a GDB listener on port 1234. 2.1 No `--help`, no `--version`: an unknown first argument is taken as the firmware file.	Validate configuration conflicts, image bounds and startup failures; add stable session identity. 2.1 Fix termination (PS11); add version and help.
Debugger 2.1	avr-gdb 15.0.50 connects and can break, step, inspect, watch and interrupt on ATmega328P and ATmega2560. EXECUTED	The defects of 22 and 25; a free port; loopback only.
Batch diagnosis	`-b` logs breakpoint hits; `-w` prints watched memory; register/SRAM dumps and tracing.	Keep these semantics; add interactive stopping and typed inspection through GDB.
Measurements	Coverage, flat profile and CALL/RET-derived call graph.	Fix flash range, paused-state accounting and interrupt attribution before making exactness claims.
UART and tests	PTY/TCP bridges and headless text input/output; output-match termination. 2.1 Input is a C string delivered one byte per millisecond whatever the baud rate (PS15).	Binary-safe I/O, deterministic scheduling, responsive capture and replay contracts.
Bootloaders 2.1	`--bootloader` loads a HEX or ELF bootloader and always starts an STK500 uploader thread.	PS08, PS13, PS18, PS19; a switch for the uploader; another protocol.
MCP	`init_arduino_project`, `compile_firmware` and `run_simulation`; stdio transport. 2.1 Three compiled copies of the server exist (MC06).	Long-lived debugger processes, structured outcomes, doctor and bounded async operations.
Agent instructions and tests 2.1	`SKILL.md` tells a model how to use the CLI; its first rule is "always use `--max-steps`". `tests/` holds two PowerShell scripts.	Keep `SKILL.md` in step with the tools; tests that run on Linux and in CI.
Protoduino	Cooperative processes/protothreads, event queue, optional inboxes, IPC pools/pipes, terminal and layout/render code.	Host decoders; fixes to cleanup and repaint cases; explicit feature fixtures. 2.1 The findings of 26 to 28.
Module loading	A compilation/packaging guide describes intended modules. 2.1 A working bootloader and module loader exist beside the library, in simulation (17).	2.1 Move it into the library; hardware; the lifecycle of L3.

Important interpretation. Current Protosim `-b` is a logging observation point, and `-s` is an instruction trace option; neither is an interactive GDB prompt. Current `-w` reads configured SRAM addresses at observation points rather than stopping on a write. Preserve these useful batch tools and name new debugger tools precisely. [S1, S2]

The MCP implementation launches a fresh simulator process for each run and waits for process output. Adding `-d` to that existing one-shot call does not create a usable interactive MCP debugger: the server must separately manage Protosim, GDB, asynchronous stop events and session state. [S4]

The TUI already has element layout, appearance signatures and a command queue. The intended direction is retained vnode/element state with incremental terminal updates; a full AVR screen-cell buffer is not a prerequisite. The inspected roadmap also discusses optional alternatives, which should remain separate design choices. [S9]

04 Functional requirements

P0: first dependable release. P1: next capability layer. P2: advanced or experimental. “Pass” always refers to a declared scenario and target, not universal correctness.

ID	Requirement	Acceptance evidence
F01 / P0	Doctor works without a firmware argument.	Readable stdout and valid JSON describe the resolved environment, checks and remediation.
F02 / P0	Preserve existing independent CLI and batch use.	Existing examples run; intentional fixes to error behaviour are documented.
F03 / P0	Provide persistent source-level debugging.	Start, break, inspect, step and resume preserve one session and matching ELF.
F04 / P0	Work directly from MCP clients.	One scripted scenario passes through both supported Claude Code and Codex configurations.
F05 / P0	Link evidence to source and build identity.	Every stop identifies ELF hash, source revision/patch, profile and source location when known.
F06 / P0	Bound execution and handle cancellation.	Unreachable breakpoint, hung firmware and lost debugger return explicit outcomes without wedging MCP.
F07 / P0	Exercise and observe UART.	Binary payloads and fragmented input produce traceable byte output and no silent capture truncation.
F08 / P0	Report failures honestly.	Crash, timeout, missing tool, unsupported feature and failed assertion remain distinguishable. 2.1 So does a normal end: a process killed by its own shutdown is not one.
F09 / P1	Decode Protoduino runtime structures on the host.	Processes, event queue, optional inboxes, IPC pipes and vnode state match raw memory. 2.1 And the module table and slots.
F10 / P1	Share target profiles across build and simulation.	Compiler, linker, startup, simulator and measurements agree on memory/clock/peripherals. 2.1 And on the boot section: its size, start and the reset vector that the fuses select.
F11 / P1	Migrate a component across targets.	Separate builds meet the same behaviour contract and target-specific resource/deadline limits.
F12 / P1	Test the terminal as observable output.	A host terminal model verifies text, cursor, style and region changes from UART bytes.
F13 / P1	Return a reproducible evidence bundle.	Another developer can rebuild and replay a declared scenario from recorded inputs and versions.
F14 / P1, P2	Support runtime native modules and expanded flash.	2.1 Split. Modules (P1): the loader's scenarios pass through Protosim and MCP, not only its own harness. Expanded flash (P2): the high-address suite passes before the capability is advertised.

ID	Requirement	Acceptance evidence
F15 / P1 2.1	Transfer a file to the simulated target with a real protocol, and reset into a bootloader.	An `.elf` sent with XMODEM-CRC arrives byte for byte; a reset during the run re-enters the bootloader; flash afterwards equals an independent load of the same file.

05 User workflows

A. First use on a development machine. The agent calls doctor before building. It learns which executable paths are selected, whether AVR tools and PlatformIO are available, which source revisions are active, and which profiles are supported. A missing compiler produces a targeted installation/configuration suggestion; a missing optional provider key does not block local MCP debugging. The agent builds a small fixture and performs a bounded GDB handshake smoke check.

B. Explain and fix a stalled process. The agent starts the failing build, replays its input, pauses at a dispatch or termination point, and inspects the scheduler list and event queue using GDB plus the matching source. It distinguishes a process waiting for input from one unable to reach its cleanup state. The resulting patch includes a regression that fails before the change and passes afterward. Reading a process object never requires adding a debug process to Protoduino. 2.1 A third case belongs here: a process that is ready but starved, because another one keeps a poll pending (PD14). The decoder has to show the poll flags and the queue, not only the list.

C. Develop generously, then reduce the target. A controller component passes first on a generous virtual Mega2560 profile. The agent rebuilds for a smaller exact MCU, adapts I/O bindings, measures flash and runtime headroom, and reruns the same externally observable behaviour tests. The terminal UI can remain on a larger target when it is outside the controller component's required contract. The system reports infeasibility rather than silently dropping required behaviour.

D. Investigate a terminal rendering defect. The agent captures UART output and the reconstructed terminal screen, inspects dirty flags and the retained element tree, and reproduces text replacement, shrinking content or node removal. It compares the intended region against the actual terminal state and fixes invalidation or transport behaviour. The host may keep a full terminal buffer; AVR firmware need only retain its tree and update metadata.

E. Develop a loader across repositories. Protoduino exports a documented module ABI; an application repository contains a small module fixture; host tools link it. The agent verifies transmission, flash placement, initialisation, entry, stop and unload. It binds symbols to loaded addresses for debugging and rejects incompatible modules with a reason. A simulator memory write that bypasses the firmware loader is a diagnostic experiment, not a passing loader test. 2.1 The first four of those steps now pass in simulation (17); stop and unload do not exist yet, and the symbol binding is a controller task.

F. Flash and load over the serial line. 2.1 The agent builds a kernel and a module, resets the simulated board into its bootloader, sends the kernel's `.elf` with XMODEM, waits for the kernel to ask for a module, sends that, and checks three things: flash equals an independent load of both files, the module's process shows up in the decoded process list, and the other processes kept their turns while it loaded.

> **Definition of a completed task**
>
> A reviewable patch, a matching build manifest, preserved acceptance criteria, passing scenario results, measured resource use and explicit remaining limitations. A successful compilation or an enthusiastic model response is not sufficient.

06 Architecture

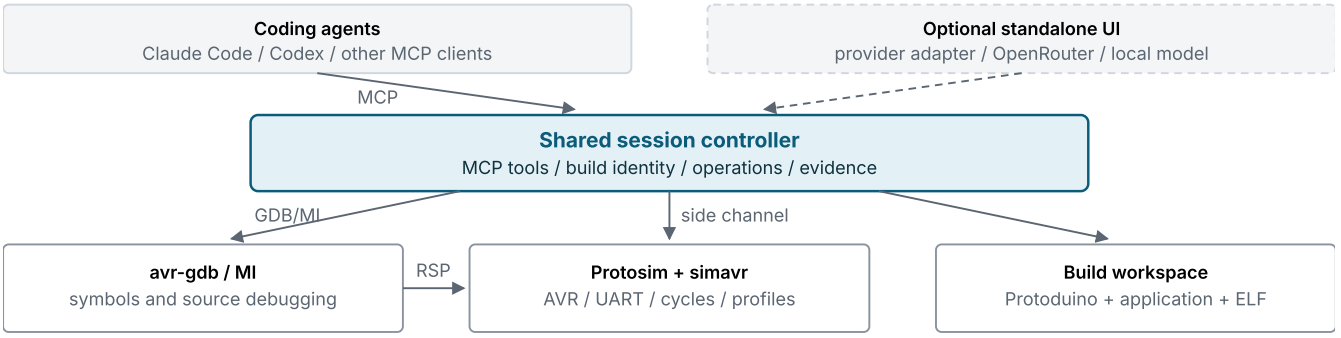


Figure 1. One controller, several entry points. Three different protocols: MCP from agent to tools, GDB/MI from controller to debugger, the GDB remote serial protocol from debugger to simulator.

Implement a reusable session/controller library behind the existing TypeScript MCP server. It manages child processes, build identities, operation state, output capture and evidence. A future standalone interface may reuse that library or connect through MCP. GDB provides source-level semantics; Protosim provides simulator-specific control and measurements. [S1, S4, R1]

Owner	Responsibility
Coding agent / optional UI	Interpret requests, inspect/edit repositories and choose experiments. Model/provider credentials stay here.
Session controller	Validate requests; serialise state-changing operations; associate profile, binary, symbols and source; retain evidence.
avr-gdb	Resolve source locations and types; manage breakpoint/step requests; read machine state through RSP.
Protosim / simavr	Execute the AVR and modelled peripherals; supply cycles, scheduled I/O, profiling and authoritative machine stop reasons.
Protoduino / application	Run the embedded program. Existing types and symbols support external inspection.

Use a narrow local control endpoint for capabilities not supplied by ordinary GDB: cycle budgets, deterministic UART delivery, counters and profile snapshots. Proposed transport: a versioned local socket or dedicated pipe, serviced on the simulator thread even while stopped. It must coordinate with GDB execution state rather than create a competing run/step owner. Model API latency is never part of the AVR instruction loop.

2.1 The side channel is also where file transfer belongs (F15). An XMODEM sender has to react to bytes from the target within the target's time, so it must run inside the simulator on simulated cycles, as the existing STK500 uploader thread does not: that thread works on host time. The harness of the loader work drives its sender from simavr's cycle timer and is a working model for it.

07 GDB implementation

Protosim already sets `gdb_port` to 1234, stops the CPU and calls `avr_gdb_init()` for `-d`. The pinned stub implements register and memory access, breakpoints, stepping and data watchpoint requests. It also has boundary and startup problems described in sections 22 and 25. [S1, S5]

What a real session did 2.1 EXECUTED

avr-gdb 15.0.50 in MI mode against `protosim -d`, firmware built with `avr-gcc -Os -g`, on ATmega328P and ATmega2560:

Request	Result
`-target-select remote localhost:1234`	`^connected`
`-break-insert main`, `-exec-continue`	`*stopped,reason="breakpoint-hit"`, function `main`, source line 14
`-data-evaluate-expression counter`	4660, the initial value of a `.data` variable: start-up code had run
`-data-evaluate-expression "(char)banner[0]"`	104 `'h'`: a constant in flash, read through the program-memory space
`-stack-list-frames`	Frame 0 with function, file and full path
`-exec-next`, `-exec-step-instruction`	`end-stepping-range` at the next line, then at the next instruction
`-break-watch counter`, `-exec-continue`	`watchpoint-trigger`, old value 4660, new value 4764, with the frame of the write
Read the last byte of flash	`^error,msg="Unable to read memory."`; the byte before it reads (AV01)
`-exec-interrupt` while running, default mode	No result record within four seconds. SIGINT to gdb stopped the target.
The same after `-gdb-set mi-async on`	`^done`, then `*stopped,reason="signal-received",signal-name="SIGINT"`

This is one firmware and one stub revision, not a qualification. It does settle the open question of edition 2.0: the existing support is enough to build on.

Startup sequence

- Resolve and freeze the target profile, ELF and source manifest. Start Protosim stopped; verify listener readiness and its actual endpoint. 2.1 Readiness cannot be taken from Protosim's output today: it prints “listening” when the bind failed (PS07). Connect and check, or fix PS07 first.
- Start avr-gdb with `--interpreter=mi2` and the matching ELF. 2.1 Send `-gdb-set mi-async on` before connecting. Connect with the remote target command. Negotiate supported commands and inspect initial state.
- Resolve requested source/function breakpoints. Report unresolved or optimised locations rather than quietly substituting another address.
- Resume only after the controller has installed input schedules, capture and execution limits. 2.1 Do not pass `--max-steps` to a debug session until PS02 is fixed: the budget runs down while the target is stopped, about 20 counts per second.

Operation	Typical MI request / behaviour
Load symbols / attach	`-file-exec-and-symbols`; `-target-select remote host:port`
Break / watch	`-break-insert`; `-break-watch`. Conditions may be evaluated by GDB on host stops.
Continue / step	`-exec-continue`; `-exec-step`; `-exec-next`; `-exec-step-instruction`
Inspect	`-stack-list-frames`; `-stack-list-variables`; `-data-evaluate-expression`; memory/register requests
Pause / stop events	`-exec-interrupt` with `mi-async on`; parse asynchronous stopped records; keep the SIGINT fallback.

Operation	Typical MI request / behaviour
More symbol files 2.1	`add-symbol-file boot.elf` for a bootloader beside the application, and one per loaded module (10).

Maintain a real MI parser for tokens, escaped strings, tuples, lists, result records and asynchronous events. Do not infer machine state from prompts or regexes over human output. Test the selected avr-gdb release rather than assuming modern host-GDB behaviour is identical. [R1]

Advertise capability results for conditional/watch breakpoints, memory spaces and maximum transfers. Simavr watchpoints observe instrumented access paths; they are not automatically a log of every host or peripheral mutation. Verify the relevant paths for each supported observation. Live variable inspection should default to a coherent stopped state.

The debugger should expose controlled writes as explicit experiments. RAM/register writes, peripheral I/O and flash programming have different semantics; a raw remote write may bypass modelled peripheral side effects. Source fixes still require rebuild and clean replay.

08 Session lifecycle

State	Allowed behaviour
starting	Allocate identifiers, start children, load identity and negotiate capabilities. Fail atomically if startup cannot complete.
stopped	Inspect consistent state; install breakpoints; queue input; start one run/step operation.
running	Capture output and service pause/cancel/status. Reject stale reads or offer an explicit pause-and-inspect operation.
faulted / exited	Preserve evidence and termination reason. Inspection is limited to what remains available.
closed	Children and endpoints are released. Retain immutable artefacts; reject execution requests.

A `debug_start` response returns `session_id`, `build_id`, `profile_id`, capabilities and `state_version`. Each accepted state mutation increments the version. Operations include a `request_id` and optional `expected_state_version` to detect conflicting clients. A single controller owns execution per simulator; parallel tests use separate instances with separate endpoints.

Budgets and stopping. Maintain separate retired-instruction, simulated-cycle and wall-clock limits. A paused GDB poll consumes no retired-instruction budget. Sleeping can advance cycles without retiring instructions. A cycle deadline stops at an instruction/event boundary; report any overshoot rather than promising impossible mid-instruction interruption. Pause/cancel remains serviceable during long runs.

Use stop reasons such as `breakpoint`, `watchpoint`, `step_complete`, `uart_match`, `cycle_budget`, `instruction_budget`, `user_pause`, `cpu_done`, `cpu_crashed`, `host_timeout` and `transport_lost`. Keep operation status separate from test verdict: reaching a UART marker does not prove all assertions passed, and reaching a diagnostic budget need not mean the tool itself malfunctioned.

Persistence and recovery. A session survives ordinary MCP calls while the controller lives. It does not automatically survive an agent exit, MCP disconnect or process restart. Default stdio mode cleans up child processes on shutdown. A future daemon mode can support reattachment with ownership and retained manifests. Reset/replay is the initial recovery path; complete snapshots require CPU, RAM, flash, EEPROM, peripheral, timer and input-queue state.

Reset is not a clean slate in simavr 2.1

"Reset and replay" needs three things that do not hold today. `avr_reset()` discards every pending cycle timer, so whatever the controller scheduled, UART input included, stops for good unless it is registered again (AV08, PS16). It restarts the cycle counter at zero, so cycle stamps and profile totals from before and after a reset do not add up (PS17). And it restarts at simavr's `reset_pc`, which Protosim never sets, so a board with a bootloader resets into the application (PS18). A replay that needs a reset should start a fresh simulator instance until these are fixed.

A memory dump is not a complete replay checkpoint. Reverse execution is not supplied merely by enabling GDB; it needs a separately implemented recording/checkpoint mechanism. Do not advertise reverse-step until its semantics are validated.

09 MCP contract

Keep `init_arduino_project`, `compile_firmware` and `run_simulation` available, with documented bug fixes. Add persistent tools using the same controller as any future standalone interface. Codex and Claude Code both document local stdio MCP connections. The MCP server does not need to invoke a model API or hold a model key. [S4, R6, R7]

Tool family	Proposed interface
Environment	`doctor(options)`, `list_target_profiles()`, `get_capabilities()`
Session	`debug_start(build_id, profile_id)`, `debug_status(session_id)`, `debug_close(session_id)`
Execution	`debug_continue(session_id, limits)`, `debug_pause(session_id)`, `debug_step(session_id, mode)`
Breakpoints	`debug_breakpoint(session_id, action, location, condition)`, `debug_watchpoint(...)`
Inspection	`debug_inspect(session_id, expressions, limits)`, `debug_backtrace(...)`, `read_memory(space, address, length)`
I/O and evidence	`uart_send(...)`, `uart_read(..., cursor)`, `run_scenario(...)`, `get_operation(...)`, `cancel_operation(...)`
Transfer and flash 2.1	`uart_send_file(session_id, path, protocol)` with XMODEM-CRC first; `reset_target(session_id, into)`; `flash_compare(session_id, elf_paths, ranges)` against an independent load of the files
Runtime adapters	`inspect_processes(...)`, `inspect_mailbox(...)`, `inspect_vnodes(...)` when the matching decoder is available; 2.1 `inspect_modules(...)`

Response and concurrency rules. Return versioned structured data plus a concise textual summary. Include capability failures explicitly, and page large structures or logs using cursors. Long work returns an `operation_id` or waits only for a bounded interval. Polling must not restart the operation. Assign event sequence numbers and distinguish host timestamps from simulated cycles. Missing cycles are null with a reason, not zero.

Use subprocess spawn/execFile with argument arrays, explicit working directories and controlled environment inheritance. Keep simulator/GDB stdout and stderr captured; only MCP protocol messages go to the server's stdout. All child diagnostics are tool results, evidence artefacts or stderr. Validate numeric bounds, paths and profile identifiers before launching.

Retry semantics matter: inspect/status are repeatable; continue, reset, UART injection and flash writes are not blindly retryable. Use request identifiers and operation lookup to recover from uncertain delivery. Serialise mutations within each session. Handle stdin closure, client termination and child failure without leaving stray simulator processes.

2.1 **Four additions from running the existing server.**

- **Never return the whole output.** `run_simulation` puts the complete stdout into `full_output`: 28,576 characters for a three-second firmware with coverage and call graph. Clients cap the size of a tool result and charge every byte to the model's context. Return a summary and a cursor; keep the log as an artefact.
- **Say what a tool does to state.** MCP tool annotations (`readOnlyHint`, `idempotentHint`, `destructiveHint`) carry exactly the retry distinction made above. Set them: inspect and status read-only, continue and UART injection not idempotent, `init_arduino_project` with overwrite destructive.
- **Declare output schemas.** A structured result with a published schema lets a client validate it; it also removes the temptation to parse text, which is how MC02 came about.
- **Judge a child by how it ended, in three parts:** exit code, terminating signal, and the stop reason the simulator reported. Node gives a signal and no code for a run that today ends normally (PS11); a controller that looks only at the code misreads it either way.

MCP is the common interface, not a promise that every client supports every optional notification/task extension. Start with ordinary tools, explicit operation polling and stdio. Add remote transport only when required by deployment. Client limits on result size and call duration differ and change; read them from the client's documentation when the tools are implemented.

10 Source and runtime inspection

The host combines live machine bytes, ELF/DWARF type information and the exact source tree. GDB maps symbols and source locations; a Protoduino adapter interprets scheduler and IPC relationships. Retain ordinary non-loadable debug sections on the host. They do not need to be flashed to the AVR. Optimised-away state remains unavailable even when debug information is present. [R2, R3]

Object	Host inspection approach
Processes	Locate the static `process_list` in `process.c`; follow bounded `next` pointers; decode state, priority, thread pointer and continuation. 2.1 Show `needs_poll` per process and the static `poll_requested`: PD02 and PD14 are visible only there.
Global event queue	Read `events`, `event_head` and `event_tail`; derive occupancy using the configured ring size and show pending destinations. 2.1 Flag a full queue: a post that fails is silent (PD03).
Optional per-process inbox	Detect `PROCESS_CONF_EVENT_INBOX` in the actual build layout; decode only if present. Default configuration disables it.
IPC pool / pipe	Follow reachable objects or known symbols; inspect free lists, counts, head/tail and buffers with bounds checks. 2.1 For a serial device process show both the pipe and the UART ring behind it: bytes strand between the two (PD15).
Vnodes / elements	Read element geometry, parent relationships, style and dirty flags; compare against the host terminal reconstruction.
Modules 2.1	Read the module table page and each slot's descriptor from flash: magic, ABI, kernel identity, process type. Compare the kernel identity with the CRC of the kernel image the controller built.
Source location	Return repository, file, line, function, instruction address and build identity. Show assembly if source mapping is unavailable.

Protoduino uses cooperative, stackless protothreads by default: `lc-switch.h` stores a line-based local continuation in a `uint16_t`. A suspended process does not have a conventional private saved call stack. Expose the continuation and process data; show the real CPU backtrace for currently executing code. An optional

virtual-thread presentation must not invent frames. [S7] 2.1 The continuation is a source line number, or an error code in the range `0xFF00` to `0xFFFE`, or `0xFFFF` for a thread that has been told to finalize. A decoder can therefore say where a suspended process waits, whether it holds a raised error it has not handled (PD21), and whether it is waiting for a cleanup that nothing will trigger (PD01), without running anything. The other continuation header, `lc-addrlabels.h`, is an `#error` at the pinned revision.

Decoder correctness. Resolve offsets from debug information instead of hard-coding one struct layout. Identify the Protoduino revision and feature flags. Bound pointer traversal, detect cycles, validate addresses against the selected memory space and report corrupt data rather than following it indefinitely. Process names and descriptors may live in program memory; distinguish AVR data addresses, flash byte addresses, code-pointer conventions and GDB's synthetic memory-space offsets.

An object with no symbol can still be discovered through a reachable pointer and known type, but arbitrary bytes do not identify their own meaning. Source-aware inspection is strongest with an unstripped matching ELF. A generic byte-inspection fallback must remain available when a semantic decoder does not support the build.

Which build can be inspected 2.1

- **Debug information is a build decision.** The session of section 07 used `avr-gcc -Os -g` directly. A PlatformIO or Arduino build is different: the Arduino AVR tool chain compiles and links with link-time optimisation, and a PlatformIO release build carries no debug information unless the project selects the debug build type. What a decoder can resolve in such an ELF, static variables like `process_list` in particular, was not measured here and must be before F09 is designed around it. The likely outcome is a declared debug profile (PlatformIO `build_type = debug`, or explicit flags without LTO) and a doctor check that the ELF has the types the decoders need.
- **One name, several objects.** `CC_EXTERN` is empty in C, so `process_current` is defined in every C file that includes `process.h` and merged by the linker (PD11). A decoder sees one symbol today; with a newer compiler the build fails instead.
- **Modules are easy.** A module of the loader work is a fully linked `ET_EXEC` file at absolute slot addresses. `add-symbol-file module.elf` registers it without section addresses; remove it on unload.

Rebuilding creates a new build identity. Do not reinterpret yesterday's running binary using today's struct definitions. Dynamically loaded modules need symbol registration and removal on unload. [R2]

11 Doctor: user contract

Proposed canonical command: `protosim --doctor`. Also accept `protosim doctor` as a discoverable alias. Neither requires a firmware positional argument. Default output is a concise, stable, readable report on stdout; `--doctor --json` emits one JSON object for agents and CI. It must report the environment it actually inspected rather than infer readiness from installed package names. 2.1 Today `protosim --help` answers “Cannot load ELF --help” and there is no version output at all, so doctor has nothing to report as Protosim's own identity until PS12 is done.

Check group	What to report
Host and binary	OS/architecture; Protosim version/revision and executable path; working directory; effective config sources.
AVR toolchain	Resolved avr-gcc/avr-g++, avr-gdb, linker, objcopy, objdump, size, nm/readelf paths and versions; timeout/error for each probe. 2.1 The compiler version matters: GCC 10 and later turn PD11 into a link error.
Build integration	PlatformIO command and selected environment; optional Make/CMake path where used; Node and MCP build entry point. 2.1 Which of the three server builds is in use, and whether it matches `index.ts` (MC06).

Check group	What to report
Dependencies	Bundled/pinned simavr identity; standalone versus submodule Protoduino revision; uninitialised or dirty dependencies.
Targets	Known MCU/profile identifiers; memory layout; supported versus experimental capabilities; compiler/profile compatibility. 2.1 Per target, whether Protoduino builds for it at all (PD17).
Debug and transport	GDB support, requested endpoint conflicts, PTY/TCP facilities and permissions. A free port check is provisional until bind succeeds.
Workspace	Repository roots and revisions; relevant source/ELF availability; linker/profile paths and stale build warnings.
Readiness	Separate ready states for run, build, debug, MCP and hardware workflows; actionable missing requirements.

Modes and behaviour. Default checks discover configuration and run bounded version/help probes; they do not install tools, change fuses, flash hardware, fetch models or modify repositories. An explicit `--doctor --check-debug` mode may create a temporary fixture, compile it, start a simulator and verify a GDB breakpoint. Report this as an active smoke test, capture its evidence and clean up. A `--timeout-ms` option caps the entire diagnosis. 2.1 The script behind section 07 is such a smoke test and takes under ten seconds; it is listed in section 29.

Optional flags: `--project PATH`, `--profile ID`, `--require build,debug,mcp`, `--json` and `--strict`. Prebuilt simulation needs no compiler; debugging a matching prebuilt ELF needs AVR-capable GDB, not a compiler. Building firmware or a smoke-test fixture needs a toolchain. A requested capability failure is an error; an unrelated optional component is a warning or skipped check.

Protosim needs no provider key when used through Claude Code or Codex; the client handles model access. For a configured standalone model adapter, report credential presence and selected provider only; never print key values, tokens or an unrestricted environment-variable dump.

12 Doctor: implementation

Parse doctor before the existing firmware-argument path. Implement a common check registry used by CLI and MCP. Each check has an identifier, dependencies, bounded probe, structured evidence and remediation. Resolve configuration in a documented order: explicit CLI/tool arguments, project configuration, user configuration, supported environment variables, then defaults. Return the origin of each resolved tool/profile choice.

```
{
  "schema_version": "1.0",
  "command": "doctor",
  "status": "degraded",
  "requested_capabilities": ["build", "debug"],
  "capabilities": {"build": true, "debug": false},
  "checks": [{
    "id": "tool.avr-gdb",
    "status": "fail",
    "required_for": ["debug"],
    "resolved_path": null,
    "version": null,
    "config_source": "PATH",
    "evidence": "Executable not found in resolved search paths",
    "remediation": "Install/configure an AVR-capable GDB"
  }],
  "exit_code": 1
}
```

Illustrative future schema and result, not an observation of the user's machine. Add `generated_at`, `duration_ms`, binary/dependency revisions, profile summaries and per-probe errors in the implementation. 2.1 On Ubuntu the remediation for this check is the package `gdb-avr`, which is what this review installed.

Contract	Required behaviour
Stdout	Text report by default; exactly one JSON value in JSON mode, including failed checks. No banner, progress line or child output outside that JSON.
Stderr	Internal diagnostics when needed; substantive check results remain in stdout. Capture child output and redact secrets.
Exit status	0: requested capabilities ready; 1: a required check failed; 2: invocation/configuration error or doctor internal failure. Warnings yield 0 unless `--strict` promotes them.
MCP doctor	Return the same schema as structured tool content. Never print a raw doctor report into MCP protocol stdout.
Evidence strength	Label found, version-checked and smoke-tested separately. Missing, timeout, unsupported and skipped are distinct statuses.

Acceptance cases include no firmware argument, missing AVR tools, paths with spaces, conflicting configs, a blocked port, stale ELF/source, a missing optional model adapter, and a probe that hangs. Validate the JSON schema and secret redaction. A doctor pass establishes environment readiness for its checks; it does not establish application correctness.

13 Target profiles

A versioned target profile is the authoritative description of CPU, memory, clock, peripherals and toolchain layout. The simulator, linker, startup code and reporting must agree. A larger host allocation alone does not give compiled AVR code access to new memory. Keep feature configuration separate from hardware capacity, and record both in the build manifest.

Profile	Flash	Usable SRAM	Status
Stock ATmega2560	262,144 B	8,192 B	Existing device model; integration tests still required. 2.1 Protoduino's timers do not compile for it (PD17).
Stock ATmega328P 2.1	32,768 B	2,048 B	The part every executed check of this edition and the whole loader work ran on. The dependable first profile.
Virtual Mega2560 / data64k	262,144 B	65,024 B	Proposed expanded data-space profile
Virtual Mega2560 / flash1m	1,048,576 B	65,024 B	Experimental; qualify high-address behaviour first
Smaller AVR	device	device	Exact supported MCU and I/O adapters required

For the Mega2560 mapping, SRAM begins at `0x0200`. Extending the last data address to `0xFFFF` yields 65,536 − 512 = 65,024 bytes, or 63.5 KiB of usable SRAM. Calling this “64 KiB RAM” without qualification is misleading: it is a 64 KiB total data address space, including registers and I/O. Stock flash/SRAM and the address map follow the device definitions. [S6, R4]

```
profile:
  id: virtual-mega2560-256k-data64k-v1
  kind: virtual
  base_mcu: atmega2560
  clock_hz: 16000000
  flash_bytes: 262144
  data_space_bytes: 65536
  sram_start: 512
  usable_sram_bytes: 65024
  linker_layout: generated-and-hashed
  feature_manifest: protoduino-full-fixture-v1
  boot:                             # new in 2.1
    section_bytes: 4096             # what the BOOTSZ fuses reserve
    reset_into: bootloader          # BOOTRST
    image: protoduino-boot-2972     # or optiboot-8.3, or none
  modules: {base: 0x6000, slots: 2, slot_bytes: 2048}
  qualification: experimental
```

Apply geometry before `avr_init()` allocates storage. Validate loadable section ranges, SRAM initialisation and stack start. Reject contradictory embedded ELF MCU/frequency metadata instead of silently overriding a selected profile. Store actual resolved configuration, not just a profile name. Existing command-line `-m` / `-f` behaviour can remain available with an explicit conflict policy. [S1, S5]

2.1 **A profile also has a boot section.** Which bootloader a board carries, how large its section is and where reset goes decide what firmware can do: on a megaAVR only code in the boot section can program flash. The loader work measured what fits where: its bootloader with the load thread offered to the kernel is 2,042 bytes on the ATmega328P, which fills the 2 KB section, and 2,972 bytes with every option on the ATmega2560, which needs the 4 KB section because the 2 KB one is too small. A profile that does not record this cannot reproduce a loader run.

Some real Mega devices support external SRAM. A matching board/XMEM configuration, pin use and wait-state model is a separate hardware profile; virtual enlarged SRAM does not automatically model those physical timings. [R4]

14 Memory engineering

Flash and code addressing. The pinned simavr has a 32-bit `flashend` and allocation based on configured size; Mega2560 already uses extended program addressing. A 1 MiB virtual flash is plausible, but is not validated here. Qualify direct/indirect calls, returns, interrupts, high-address constants, ELPM and self-programming. Define implemented RAMPZ/EIND bits and boot-region rules for the virtual profile rather than inheriting physical-device assumptions blindly. [S5, S6]

AVR GCC uses conventions for indirect calls, EIND and linker trampolines. Conventional function-pointer behaviour is not equivalent to a freely widened C pointer. Keep code-pointer/address conversions explicit in the loader and debugger. Near flash reads require appropriate placement or far-access alternatives; inspect the link map. [R5, R8] 2.1 The loader work met both limits in practice. In the ATmega2560's boot section a table in flash lay beyond the reach of `LPM`, and the bootloader refused every file until the table was read through `ELPM`. And a module's process type must lie in the first 64 KiB, because `process_new()` reads it through a near pointer.

Data space and C assumptions. The pinned core uses a 16-bit `ramend` and classic data-address semantics. Flat 256 KiB or 1 MiB SRAM is outside the initial design: banking or a different architecture would require software support. Expanding within 64 KiB still exposes 16-bit signed arithmetic assumptions, pointer casts, size counters, allocation limits and startup stack configuration. Audit Protoduino's `free_mem()` and process descriptor sizes before a full-feature profile is accepted. [S5, S7, S10]

Metric	Definition and caveat
Flash occupancy	Loadable code, constants and initialised data image, plus reserved boot/module regions. Exclude host debug sections.

Metric	Definition and caveat
Static SRAM	Allocated `.data`, `.bss` and other sections including queues, buffers and runtime objects. 2.1 And `.noinit`: the loader's bootloader keeps its only variable there.
Dynamic use	Pool occupancy, retained vnode/text storage and heap use if enabled; do not double-count allocations already in static storage.
Stack reserve	Record observed minimum SP/high-water evidence under scenarios. This is observed coverage, not a proof of worst-case stack usage.
Timing	Instruction/cycle measurements for the modelled CPU and peripherals; separate sleep/interrupt time and host elapsed time. 2.1 State what the model leaves out: simavr programs a flash page in no time, a chip takes 7 to 9 ms with interrupts off (AV05).
Fairness 2.1	Turns a reference process gets, and its longest wait, while a scenario runs. The loader work measured 18,053 turns and a longest wait of 15.1 ms during a 350 ms module load, and 2.0 ms with one scheduler function changed (PD14).

Define a full-feature fixture that actually invokes scheduler, IPC, Unicode and TUI paths and includes representative buffers and texts. Merely enabling configuration flags can leave unused code removed by the linker. Establish measured flash, static RAM and runtime headroom before claiming that 256 KiB flash or a 63.5 KiB SRAM profile is sufficient.

Expanded SRAM does not require an AVR framebuffer. Budget nodes, text, layout/damage metadata, queues, transport buffers and stack for the intended terminal design.

15 Target migration

The AI can develop with headroom and then search a measured target/configuration space. This is a rebuild-and-test workflow: changing `-m` does not turn a Mega2560 executable into ATtiny firmware. Different instruction sets, peripherals, interrupt vectors, clocks and ABI assumptions must be accounted for. The candidate set is the intersection of toolchain support, compiled simavr cores and available platform adapters. 2.1 And of what Protoduino itself compiles for: at the pinned revision its clock header accepts the ATmega328, 168 and 88 families only (PD17).

Stage	Required output
Freeze behaviour	Protocol examples, state transitions, error cases, deadlines and required/optional features; record the assertion set.
Establish baseline	Passing scenarios and resource report on an explicit development profile.
Select candidate	Exact MCU identifier, profile revision, peripheral mapping and target-specific build flags.
Build and inspect	Link map and section sizes; diagnose over-budget code, buffers, missing peripherals or unsupported instructions.
Optimise	Consider smaller queues, compact data, reduced diagnostics, shared state and algorithm changes. Record tradeoffs.
Replay and compare	Check equivalent observable behaviour with target-specific timing limits and transports.
Rank qualified targets	Report flash/RAM headroom, observed stack reserve, latency and unsupported physical aspects. Reject infeasible choices explicitly.

Test abstraction. Express tests as logical input/output actions, then map them to each target: a serial packet on a Mega may become a test-harness input to a pure controller function on a device without the same UART. State clearly when the transport differs. A component-only success is not a success for the complete terminal-enabled application. Use target-specific deadline assertions rather than expecting identical instruction counts. Preserve input ordering, required results and failure behaviour. Sweep boundary values and queue pressure before reducing capacities. Select headroom policies explicitly; a minimal linkable image with no runtime reserve is not a deployable result.

Evidence for design decisions. Produce a candidate matrix with exact builds and rejected-target reasons. Keep acceptance tests stable while the model modifies implementation. Changing a required feature or relaxing a deadline is a requirements decision, not an optimisation. Report “smallest among tested candidates” unless the search exhaustively covers the defined candidate space.

2.1 **A worked instance exists.** The loader work builds one bootloader source for four parts and fills each boot section with as many options as fit, by building and measuring: 2,042 of 2,048 bytes on the ATmega328P, nothing in 2 KB on the ATmega2560, everything in 4 KB. Its `fit.sh` is a small resource-frontier search of the kind this section describes, and its result table states which candidates were rejected and why.

16 Vnode terminal UI

The intended AVR TUI retains a tree or arena of elements, text references/content, layout state and invalidation metadata. Rendering emits ANSI/VT cursor, style, text and clear operations over UART. The terminal holds the displayed screen. A host reference terminal may maintain a full screen buffer for testing, without imposing that memory requirement on the AVR. [S9]

Concern	Required design behaviour
Identity and lifetime	Stable element identity; explicit ownership and lifetime of referenced strings and scroll buffers.
Invalidation	Detect text edits, property changes, insertion/removal and movement. Retain enough old geometry or repaint an enclosing region.
Overlap and clearing	Clear vacated cells, redraw exposed siblings/backgrounds and define z-order and clipping.
Transport	Handle partial writes and backpressure without losing UTF-8 bytes or control sequences. Retain progress across yields if required. 2.1 Output that reaches the pipe is not yet output on the wire: the serial device process leaves bytes in its pipe until it is polled again (PD15), and the UART ring behind it is 16 bytes (PD12).
Unicode	Choose and document supported display-width semantics. Bytes, Unicode code points and terminal columns are different quantities.
Host testing	Parse emitted VT bytes, reconstruct the terminal, compare regions/cursor/style and retain evidence.

The inspected implementation uses 16-bit appearance signatures and a command queue. Text signatures include pointer and length rather than text content. Removal of trailing elements is not detected by the shown damage pass when remaining geometry is unchanged. Direct box-character writes ignore partial pipe writes. These are concrete review items, not reasons to adopt an AVR framebuffer. [S9; PD05 to PD07]

Required scenarios. Replace “ON” with “NO” in the same buffer; shrink text; clear an entire tree; remove the last child; move and resize an overlay. Render while the TX pipe has one byte free; split a three-byte box glyph; saturate and resume the renderer; verify no output was discarded. Exercise nested clipping and parent repaints; test multibyte text, invalid UTF-8, wide and combining characters according to the selected policy. Verify that an unchanged frame emits no unnecessary redraw and that a full resynchronisation can recover after a terminal reconnect.

One rune per cell is an explicit limitation of the current clipping helper, not general Unicode terminal correctness. Full grapheme shaping is not implied. The earlier 13,440-byte screen-cell estimate is not a requirement for this architecture.

17 Native modules 2.1

There are two different deliverables: a bootloader that accepts a linked image and a runtime module loader that manages independently compiled code. ELF32 describes a file format, not a 32-bit CPU requirement. Edition 2.0 planned both in four stages and noted that the existing guide, `docs/loader.md`, is not an executable design

(PD10). Since then a design with running code exists. This section is rewritten around it; the stage plan is kept as the frame.

Stage	Scope of edition 2.0	Status
L0 whole firmware	Continue using the normal compiled-image load/reset path. Do not describe this as live module replacement.	A serial bootloader takes the unmodified `.elf` as a stream, over XMODEM-CRC from any terminal program. It checks the file from its headers, refuses before it erases, and makes an image valid only by a record written last. 2,014 bytes on the ATmega328P. 13 scenarios pass in simavr, including a 106 KB image on the ATmega2560. **EXECUTED**
L1 host-linked package	Link for a reserved flash slot on the host; include initialised data, zero-fill requirements, entry and ABI metadata. The device validates and writes the image.	Done, with the ELF file as the package. A module is a Protoduino process type linked for a slot against the symbols of one kernel build; its first bytes are a descriptor with magic, ABI version and the kernel's identity. 9 scenarios pass on the unmodified scheduler, also with a stock Optiboot 8.3 as the bootloader. **EXECUTED**
L2 bounded ELF object support	Accept selected `ET_REL` inputs only after defining supported sections, relocation kinds, import resolution, alignment and rejection behaviour.	Not done and argued against: linking on the device is what made Contiki's loader large and is why Contiki-NG removed it. Proposed in its place: slots aligned to 512 bytes and a sorted list of high-byte fix-ups applied as the stream passes, so one file loads into any slot without a relocator. **PROPOSED**
L3 lifecycle / replacement	Quiesce owned processes, callbacks, timers and messages; remove stale references; commit replacement only after validation.	Partly: a module table written after validation, start through `process_new()`, restart after reset, a second module beside a running one. Missing: stop, unload, replacement, and everything PD01, PD02 and PD04 stand in the way of.

On the classic Mega-style target, native instructions execute from program flash. A RAM buffer may stage incoming bytes but cannot become ordinary native instruction memory. A real loader needs the applicable self-programming path and boot-region permissions. Direct simulator flash modification is useful for diagnosis but does not test that device path. [R4]

How the loader meets the package and lifecycle contract of edition 2.0

Contract item	In the loader	Met
Format/version	The ELF header itself, checked by 17 fixed-value rules; module magic and ABI byte	yes
Target-profile compatibility	Core family from the ELF flags against the build; optionally the device signature	yes
Runtime ABI identifier	CRC-16 of the kernel image, computed by the kernel and carried by the module; a mismatch is refused before the first page is programmed	yes
Section layout, entry point	Program headers; the entry point is the address of the descriptor	yes
Data initialisation, zero-fill	A module has no static data. Its state is the process struct the kernel allocates; the module linker script fails the build otherwise.	by exclusion
Imports/exports	Resolved on the host against the kernel's symbol addresses; nothing on the device	yes
Integrity check	CRC-16 per XMODEM block in transit; CRC of the firmware image in its record. No checksum of a module is kept after programming.	partly
Bounds and protected regions	A window per load: a module can touch its slot only; the kernel image was verified unchanged after every load	yes

Contract item	In the loader	Met
Authenticity	Not designed; one descriptor byte is reserved for it	no
States validated, staged, programmed, initialised, running, stopping, unloaded/failed	Refused, programmed, listed in the module table, running. No stopping or unloaded state.	partly
Symbols registered with GDB on load, removed on unload	Not done; straightforward (10)	no

What it adds beyond the contract. The load is one Protothreads v2 thread that does no I/O. The bootloader schedules it at reset; a Protoduino process schedules the same code, where it sits in the boot section, while the system runs. Loading therefore does not stop the scheduler: a heartbeat process took 18,053 turns during a 350 ms load. And the same thread loads through Optiboot's `do_spm` when the kernel links its own copy, so a board keeps its bootloader.

What simulation cannot show about a loader

The loader goes through the firmware path, as edition 2.0 demands. That is still not hardware evidence, for a reason edition 2.0 does not name: simavr executes SPM from anywhere, replaces a page without an erase, has no read-while-write lockout and programs in no time (AV02 to AV05). A loader that forgot the erase, forgot to re-enable the application section, did not wait, or ran outside the boot section would pass every one of the 22 scenarios. A strict flash mode in Protosim that reports those four would turn blind spots into test failures; it is the most valuable single addition for stage G5.

Full design, measurements and code: `protoduino/docs/elf-loader-design.pdf` and `protoduino/docs/elfloader/`. The code sits beside the library at the paths it would have in it; no library source file was changed.

18 Verification and evidence

A reproducible experiment bundle. Capture repository commits and dirty-patch hashes; dependency pins; compiler/linker/GDB versions; flags and feature configuration; profile/linker-script hashes; ELF and loadable-image hashes; scenario version; initial memory/peripheral policy; input payloads with actual delivery cycles; stop events; assertions; terminal capture; and resource measurements. Large traces remain artefacts; the model receives focused excerpts and references.

Suite	Minimum release evidence
Doctor	Schema-valid stdout; missing tools and timeouts; config precedence; secret redaction; clean temporary smoke-test teardown.
Termination 2.1	Every way a run can end, on every UART mode: exit code, signal and reported stop reason agree, and none of them is a signal the process sent itself.
Debugger	Connect, breakpoint, condition, watch, step, pause, inspect and teardown; malformed/lost responses; address boundaries and optimised-code behaviour.
Instrumentation	No counts while stopped; correct instruction/cycle distinction; high-flash coverage; interrupt and nested call accounting.
MCP	Existing batch compatibility; persistent state; cancellation; concurrent requests; argument handling; client disconnect and process cleanup.
Protoduino	Queue full/empty/wrap; cleanup with no incoming events; poll traversal under finalisation; message ownership and restart generations. 2.1 Events under continuous polls; output of the serial process without further polls.

Suite	Minimum release evidence
TUI	Content mutation, removed elements, old bounds, partial output and width policy; compare UART reconstruction with expected terminal state.
Migration / loader	One behaviour contract on two qualified profiles; loader rejection and recovery; high-address suite before 1 MiB qualification. 2.1 The loader's T1 to T13 and M1 to M9 through Protosim and MCP.

2.1 **An independent reader as oracle.** The strongest check in the loader work costs one line: after the firmware under test has written flash, compare it with what simavr's own ELF reader produces for the same file. Two implementations that agree on 106 KB of image are unlikely to share a bug. Protosim should offer it as `--expect-flash <file.elf>` and as the `flash_compare` tool; it needs honest exit codes to be of use in CI.

Replay reproducibility requires controlling external inputs. Record the cycle when each byte/event is applied, not only host arrival time. Queue host input while paused according to policy. Seed any randomness and define initial RAM/EEPROM values. Different host scheduling or peripheral models may prevent bit-identical replay; report the declared reproducibility level.

Compiler success, debugger stop status and test verdict are separate records. Use reference fixtures and preserved assertions to avoid a model “fixing” the tests to accept a regression. Resource results identify scenario coverage; they are not proof of all possible execution paths. Hardware measurements remain a separate qualification layer.

Performance metrics should separate build duration, simulated cycles per host second, tool latency, trace volume and model tokens. Publish task completion rates only after a reproducible benchmark protocol and baseline exist.

19 Providers and hardware

Provider independence. For Claude Code and Codex, the coding client already manages model inference, context and source edits. Protosim MCP exposes deterministic tools and evidence. A future custom client can use OpenRouter, direct APIs or a local model adapter; it should reuse the controller contract. Named models are user examples, not fixed implementation dependencies or promises of availability through a particular endpoint. A standalone provider adapter should declare tool-call support, context/output limits, cancellation and retry behaviour. It records the requested/resolved model identifiers and usage where exposed. It must not silently repeat state-changing tool operations after uncertain delivery. Model-specific prompting stays outside simulator semantics and acceptance tests.

Physical AVR debugging. A supported device and probe may expose on-chip debug access without adding a firmware agent. The chosen GDB-compatible server bridges to the hardware interface. Interface/probe/device support must be verified per target; JTAG, debugWIRE and UPDI cover different AVR families and configurations. [R9]

Aspect	Simulation	Physical target
Memory access	Host-owned modelled state; explicit stopped snapshots are straightforward.	Depends on the probe, MCU and halt behaviour.
Time and environment	Can schedule virtual inputs and pause the modelled machine.	External equipment continues while the CPU is halted; peripheral behaviour varies.
Breakpoints and trace	Software-managed observations within implementation limits.	Hardware resources and transport rates may be limited.
Flash programming 2.1	Permissive and instantaneous (AV02 to AV05).	Boot-section permission, erase before write, read-while-write lockout, 3.7 to 4.5 ms per erase and per write, lock bits and fuses.
Expanded profiles	Can explore virtual memory extensions after qualification.	Must match actual device/board memory and timing.

Aspect	Simulation	Physical target
Reproduction	Reset plus a recorded virtual input scenario.	Requires fixtures and control over physical inputs; simulator replay alone is insufficient.

Reuse the source/ELF decoders over a different memory-access backend where possible. Where hardware debugging is unavailable, an optional firmware stub or telemetry channel is a different integration requiring firmware changes. This does not alter the no-firmware-agent design for simulator inspection.

Qwenputer inspired the idea of coupling reasoning with execution. This toolbox executes AVR firmware externally in simavr. An AVR compiled into model weights or a MicroPython AVR port is independent research, not a prerequisite for the proposed development loop.

20 Delivery roadmap

Gate / suggested owner	Scope	Exit evidence
G0: trustworthy baseline Protosim + runtime maintainers	Pin repositories, reproduce findings, correct error reporting and measurement defects; define baseline fixtures. 2.1 In this order: termination (PS11), exit codes (PS01), the MCP failure envelope (MC03, MC05), the step budget under GDB (PS02), the listener (PS07). Then the scheduler fixes PD01, PD02, PD03 and PD14, which every later gate stands on.	Known failures have regression cases. Baseline manifests are reproducible. 2.1 The checks of section 29 run in CI on Linux.
G1: environment doctor CLI/controller owner	Check registry, configuration resolution, text/JSON modes, capability readiness and optional debug smoke test. 2.1 With `--version` and `--help`.	New checkout diagnosis identifies actionable blockers; JSON remains valid on failure.
G2: persistent GDB/MCP Controller + simulator owners	MI parser, session lifecycle, bounded operations, UART, matching symbols and cleanup.	Claude Code and Codex complete the same breakpoint/inspect/resume workflow.
G3: source/runtime evidence Protoduino + controller owners	Process/queue/pipe/vnode decoding and replay bundles; TUI defect regressions. 2.1 A measured answer to which builds carry usable debug information (10).	Host decoded state agrees with raw memory and produces a reviewable source fix.
G4: profiles and migration Toolchain/simulator owners	Stock profiles; qualified data64k profile; full-feature fixture; candidate comparison. 2.1 Protoduino building for the ATmega2560 (PD17); boot sections in profiles.	Two targets pass a shared component contract with measured budgets.
G5: runtime modules Protoduino + host tooling owners	Host-linked package, ABI, flash programming and lifecycle; debugger symbols. 2.1 L0 and L1 exist. Remaining: binary transfer and reset in Protosim (F15), the strict flash mode, unload and replacement, module symbols in GDB, hardware.	Load/run/stop/unload and negative recovery cases pass through the firmware path. 2.1 And on an Uno and a Mega.
G6: advanced options Explicit research owners	1 MiB flash, bounded on-device ELF relocation, snapshots, hardware backend, optional standalone UI. 2.1 Or the fix-up stream in place of on-device relocation.	Each feature has its own capability qualification and unsupported-case policy.

Recommended scope of the first dependable release. Deliver doctor, truthful batch outcomes, persistent GDB sessions, basic UART interaction, source-linked results and one complete MCP demonstration on a stock MCU. Keep virtual-memory engineering as a parallel workstream only after baseline instrumentation is trustworthy. The agent workflow can be useful before the most ambitious memory and loader work is finished. 2.1 Make the stock MCU the ATmega328P: everything in this edition ran on it, and Protoduino builds for it.

Do not estimate completion dates solely from this document. Owners should validate toolchain availability and the defect reproductions, then estimate bounded milestones. The current MCP TypeScript code makes a TypeScript controller a practical default, subject to team review; GDB and simavr remain separate native

components.

21 Design decisions

Decision	Recommendation / rationale
Execution ownership	One controller owns GDB and simulator control per instance. A second UI attaches through it, not as an independent command source.
Simulator side channel	Small versioned local endpoint for budgets, scheduled I/O and metrics; keep GDB for source debugging.
Session lifetime	Start with stdio-owned child processes and clean shutdown. Add a reattachable daemon only when a demonstrated workflow needs it.
Target qualification	2.1 Stock ATmega328P first, then stock Mega2560 once Protoduino builds for it; virtual data64k next; 1 MiB flash experimental until its boundary suite passes.
TUI correctness	Explicit dirty/content-generation and removal invalidation; bounded metadata, no mandatory AVR framebuffer.
Module scope	2.1 Host-linked ELF in a slot, as built. Decide between the fix-up stream and an on-device relocator before either is started; decide whether a module checksum is stored.
Protocol compatibility	Version schemas; expose capabilities; preserve existing tools and CLI behaviour except clearly documented bug fixes.
Error semantics	Separate tool failures, simulator stop reasons and test assertions. Never infer correctness only from exit code or UART text.
One server build 2.1	Build the MCP server in one place and stop committing compiled copies; let the package bundle from that build.
Simulator fidelity 2.1	Fix the deviations that hide firmware bugs (AV02 to AV05) behind a strict mode in Protosim rather than patching the pinned simavr, and record the mode in the evidence bundle.

Ambitious additions with concrete value. Failure minimisation: reduce a failing input trace to the shortest reproducer while preserving the assertion failure. Resource-frontier search: explore target/configuration combinations and report measured Pareto tradeoffs. Runtime explanation: join process/queue snapshots with the source and terminal at the same stop. Cross-build comparison: highlight changed behaviour and budgets for the same scenario. These are proposed extensions, not current capabilities.

Success measures. Measure time from a clean checkout to the first successful debug session; percentage of scenario runs with complete provenance; reproducibility of failures; verified fix rate on a curated defect set; resource savings without requirement loss; and rate of incorrectly reported success. Compare AI-assisted and manual workflows using the same fixtures, environment and completion criteria. 2.1 The curated defect set now has 57 entries with a trigger each (22 to 28).

The product goal is a distinctive AI toolbox. The defensible claim is the combination of capabilities and measured outcomes; “one of a kind” should remain a positioning ambition until supported by comparative research.

22 Findings: Protosim execution and measurement

Priorities are engineering recommendations. The status says how the finding was established: EXECUTED for this edition, ISOLATED by a host check of edition 2.0, SOURCE by reading. Source paths refer to the pinned revisions of section 02. Findings PS01 to PS10, MC01 to MC04, AV01 and PD01 to PD10 are those of edition 2.0, with their wording kept and their status updated; the others are new.

ID	Effect	Action and regression	Status
PS01 high	**CPU failures can exit successfully.** `cpu_Crashed` breaks the loop, but the final return expression returns 0 in both branches. An agent or CI job can classify a crash as success. `src/protosim.c:403–406, 470`	Centralise termination state and map faults to nonzero exits. Preserve stop reason separately from assertion verdict and diagnostic budget completion. *Regression:* invalid-opcode/crash fixtures give a structured `cpu_crashed` and a failing process status; a benign bounded stop is identified correctly. 2.1 Holds for headless runs; see PS11 for the others.	EXECUTED "CPU CRASHED", exit 0
PS11 high 2.1	**A normal end is a signal death.** With the default UART bridge, `uart_pty_stop()` calls `pthread_kill(thread, SIGINT)`. No handler is installed, so the signal terminates the whole process. Every run that ends while the bridge is active is affected, whether on the step limit, CPU done or CPU crashed. A run with `--uart0-out` or `--exit-on-uart` has no bridge and ends normally. A shell sees status 130; Node sees a signal and no exit code; a bash script that runs Protosim is itself ended by the shell's SIGINT handling. `src/uart_pty.c:210`	Stop the bridge thread without a process-directed signal: a flag and a wake-up pipe, or `pthread_cancel` at a blocking point. Then fix PS01. *Regression:* for each ending and each UART mode, the exit code is the documented one and no signal is reported.	EXECUTED step limit: killed by signal 2; the same with `--uart0-out`: exit 0
PS02 high	**Paused GDB calls pollute instrumentation.** `avr_run()` can return while stopped without executing an instruction. The outer loop still counts a step and calls `profiling_step()`. Coverage can be marked for unexecuted code; a CALL opcode can repeatedly push callgraph frames during a pause. 2.1 It also spends `--max-steps`: the stub polls every 50 ms while stopped, and each poll counts. `src/protosim.c:393–408`; `sim_avr.c:347–357`; `profiling.c:152–185`	Instrument actual instruction retirement separately from polling, sleep and interrupt service. A cycle delta alone is not a complete retirement signal. *Regression:* pause on a CALL for repeated debugger polls; require unchanged instruction count, coverage, call depth and function call count; test sleeping and interrupt-only advancement too.	EXECUTED `--max-steps 700` fired after 18.3 s stopped at a breakpoint
PS03 high	**Coverage and PC profiling stop at 32 KiB.** `FLASH_WORDS_MAX` is 32768/2. Allocation and updates cap per-address coverage/cycles even when the simulated device has larger flash. `include/protosim_core.h:20`; `profiling.c:131–163`	Allocate by actual flash geometry, retain allocation lengths, use those for every bound and report allocation/truncation failures. *Regression:* execute known functions below and above `0x8000` and at the last valid instruction; compare totals against the scenario.	SOURCE

ID	Effect	Action and regression	Status
PS04 high	**Callgraph attribution needs interrupt semantics.** The stack tracker treats RETI like RET but does not push an interrupt frame at interrupt entry. An ISR return can pop a foreground call frame. Post-run PC may also reflect interrupt dispatch. `include/protosim_core.h:55` ; `profiling.c:165–195`	Track interrupt entry/exit explicitly; distinguish instruction destination from post-interrupt PC. Qualify tail calls and unresolved functions in reports. *Regression:* interrupt a nested foreground call chain, return with RETI and verify intact caller frames and separate ISR cycle totals. 2.1 A related oddity seen here: the hottest edge of a run was reported as `__do_clear_bss → main` with 0 calls and 99.75 % of the time; code that falls through into `main` is booked as a caller.	SOURCE not re-run
PS12 medium 2.1	**No help and no version.** The first argument is always the firmware. `protosim --help` prints “Cannot load ELF --help”; there is no way to ask the binary what it is.	`--help` , `--version` with the git revision and the pinned simavr; parse them, and doctor, before the firmware argument.	EXECUTED

23 Findings: Protosim image, memory, startup, input

ID	Effect	Action and regression	Status
PS05 high	**Unvalidated debug ranges and HEX copy.** Watches and SRAM dumps index `avr->data` without validating the requested range. The main HEX path copies at `hex_base` without checking fit against selected flash. Invalid ranges can access host memory outside the modelled region. `debug.c:56–85` ; `protosim.c:239–243`	Validate address spaces and complete ranges with overflow-safe arithmetic before reads/copies. Reject invalid size, malformed numeric input and incompatible images. *Regression:* the final valid byte, a crossing range, huge length, negative text and an image larger than the MCU give bounded errors, not a host crash.	EXECUTED `--dump-sram 0x7ff00000:16` : killed by signal 11
PS06 medium	**Incorrect SREG diagnostic.** `dump_registers` passes `a->sreg[0]` to a function expecting a packed status byte. simavr stores eight separate flag bits; the dump therefore loses flags other than the first bit. `debug.c:32–52` ; `sim_avr.h:260`	Pack all eight simavr flag entries, or use a verified helper that returns packed SREG. *Regression:* set I, Z and C independently and together; compare the CLI dump, GDB register value and core state.	SOURCE
PS07 high	**GDB startup success is not checked.** `avr_gdb_init()` can return −1, but Protosim still prints that the server is listening. The port is fixed at 1234. The pinned stub binds a zero-initialised IPv4 address, so its listener is not restricted to loopback. `protosim.c:315–319` ; `sim_gdb.c:951–1003`	Check the return value; make endpoint selection explicit; bind loopback by default; report actual readiness. Allocate separate endpoints for concurrent sessions. *Regression:* occupy the port, launch `-d` and require an immediate startup error; verify the bound address and two independent sessions.	EXECUTED second instance printed “listening”; the port accepted a connection on a non-loopback address
PS08 medium	**ELF bootloader overlay replaces the whole flash.** The ELF overlay copies all temporary-core flash bytes, including erased areas, over the app image. `bl_size` stays zero in this branch, so the `codeend` update does not use the actual bootloader extent. `protosim.c:295–310`	Define whole-image versus overlay semantics; for an overlay copy validated loadable ranges only and track the real extent. Release temporary firmware/core resources. *Regression:* app and bootloader in disjoint regions: preserved app bytes, correct boot extent and repeatable startup.	EXECUTED with an ELF bootloader only its banner appeared, the application's never

ID	Effect	Action and regression	Status
PS13 medium 2.1	**An ELF bootloader is started at address 0.** Its entry is read from the temporary core's program counter, which simavr never sets when it loads a file. With PS08 the result looks right by accident: address 0 is erased by then, and erased flash runs through to the bootloader. `protosim.c:304`	Take the entry from the lowest loaded address of the bootloader or from the ELF header. *Regression:* the same bootloader as HEX and as ELF reports the same entry.	EXECUTED `entry=0x0000` as ELF, `0x7800` as HEX
PS14 medium 2.1	**A HEX firmware with two regions loses the second.** The main firmware is read with simavr's single-chunk reader. `protosim.c:195`	Use the multi-chunk reader, or Protosim's own once PS09 and PS10 are fixed. *Regression:* application and bootloader in one HEX file both arrive.	EXECUTED "Loaded HEX: 180 bytes", bootloader absent
PS15 high 2.1	**UART input is a C string at one byte per millisecond.** `--uart0-in` cannot carry a zero byte, so no binary file. Bytes are injected every millisecond from 1 ms after start, whatever the baud rate and whether or not the receiver is enabled: early bytes are lost, and at 9600 baud they arrive faster than a byte time. `protosim.c:84, 349`	Input from a file or pipe; start when the receiver is enabled, as the TCP path does; pace from the configured rate on simulated cycles. This is the deterministic UART delivery of sections 06 and 18. *Regression:* a binary payload arrives intact at three baud rates.	EXECUTED receiver enabled at 5 ms: the first 5 of 16 bytes lost
PS16 medium 2.1	**Input stops for good after a reset.** simavr's reset discards the cycle timer that drives the injection (AV08), and nothing registers it again. `protosim.c:349`	Register again after a reset. *Regression:* input continues across a watchdog reset.	EXECUTED
PS17 low 2.1	**Profile totals break across a reset.** The cycle counter restarts at zero, so totals no longer match the per-function sums.	Keep a running total in Protosim. *Regression:* percentages stay within 100 after a reset.	EXECUTED "hottest single edge: 200.0 %"
PS18 medium 2.1	**A reset does not re-enter the bootloader.** simavr's `reset_pc` is never assigned, so a reset during a `--bootloader` run restarts at address 0. An application that asks for an update by resetting cannot be tested.	Set `reset_pc` when a bootloader is given. *Regression:* a watchdog reset lands in the bootloader.	SOURCE a harness that sets it resets correctly
PS19 medium 2.1	**The STK500 uploader always runs, with a fixed page.** `--bootloader` starts the uploader thread unconditionally; no switch turns it off or selects another protocol, so another bootloader cannot be tested without it interfering. Its page size is fixed at 128 bytes, which suits ATmega328P-class parts only. `protosim.c:384` ; `include/stk500_uploader.h:36`	A switch for none / STK500 / XMODEM; page size from the part. *Regression:* the loader scenarios of section 17 run inside Protosim.	EXECUTED SOURCE

24 Findings: MCP server

ID	Effect	Action and regression	Status
MC01 high	**Shell command construction is fragile.** The server interpolates environment names and joins executable paths, firmware paths, breakpoints and free-form flags into shell strings. Paths with spaces split incorrectly; metacharacters can be interpreted as shell syntax. `mcp-server/src/index.ts:164–165, 218–235`	Use spawn/execFile with argument arrays and explicit cwd. Validate profiles and structured flags instead of accepting unchecked command fragments. *Regression:* build/run from a directory containing spaces and literal shell metacharacters; verify exact arguments and no unintended command execution.	EXECUTED a firmware path with a space: “Cannot load ELF” the first half
MC02 medium	**Output parsers do not match current text.** The breakpoint regex expects a summary format the current loop does not emit. The hot-path regex is over-escaped. `index.ts:239–257`; `debug.c:92–103`; `profiling.c:532`	Replace regex-derived telemetry with versioned structured simulator results. Preserve human text only as supplemental evidence. *Regression:* accurate breakpoint hit counts and a populated hot path for fixtures known to produce them.	EXECUTED breakpoint hit, call graph printed: `"breakpoints_hit": {}`, `"hot_path":null`
MC03 high	**Execution errors lack a failure envelope.** `run_simulation` catches subprocess errors but returns ordinary content without `isError`. The outer error message escapes template interpolation and can show literal placeholders instead of the operation and error. `index.ts:276–302`	Set tool failure status consistently; include exit code, signal, stop reason and bounded stdout/stderr. Fix interpolation and preserve the original error context. *Regression:* missing executable, invalid input, CPU crash and timeout are each distinguishable from a successful run.	EXECUTED “protosim: not found” returned with `isError` false
MC04 medium	**Project initialisation overwrites existing files.** `init_arduino_project` writes `platformio.ini` and `src/main.cpp` even when they exist. `env_name` changes the environment label while board remains `uno`. `index.ts:114–151`	Default to creating missing files only; require an explicit overwrite choice. Accept/validate board or profile separately from environment name. *Regression:* initialisation on an existing project preserves its contents; a Mega profile changes the actual board setting.	EXECUTED a Mega project came back as `[env:mega]` with `board = uno` and the template sketch
MC05 high 2.1	**The documented way to run returns no result.** `max_steps` is a required argument. A run that ends on it with the default UART bridge is killed by SIGINT (PS11); Node rejects, and the tool returns “Execution completed or errored” followed by raw text, with `isError` false and none of the parsed fields. Only a headless run, with `--uart0-out` or `--exit-on-uart` among the flags, reaches the parser.	Fix PS11. Until then pass `--uart0-out` from the tool, and judge the child by code, signal and reported stop reason together (09). *Regression:* a step-limited run returns the structured result with stop reason `instruction_budget`.	EXECUTED
MC06 medium 2.1	**Three compiled servers.** `tsc` writes `mcp-server/build/index.js`. The repository also tracks `mcp-server/dist/index.js`, a 716 KB bundle, and `mcp-server/src/index.js` with maps and declarations. `packaging/nfpm.yaml` installs the bundle as `/usr/lib/protosim/mcp/index.js`, started by `/usr/bin/protosim-mcp`. Nothing ties any of them to `index.ts`.	One build output; bundle from it at packaging time; remove the committed copies; let doctor report which one runs. *Regression:* the packaged server and a fresh build answer `tools/list` identically.	EXECUTED both builds started and listed the same three tools; SOURCE

ID	Effect	Action and regression	Status
MC07 medium 2.1	**Unbounded in both directions.** The whole stdout of a run is returned in `full_output`: 28,576 characters for a three-second firmware. And the child is run with Node's `exec`, whose default buffer is 1 MiB: a longer output, an instruction trace for instance, kills the child and lands in the MC03 path. With `max_steps` 0 the call never returns.	Stream to an artefact, return a summary and a cursor; a wall-clock limit on every call; reject a non-positive `max_steps`. *Regression:* a trace of a million instructions returns a bounded result and a readable log.	EXECUTED size, and no answer in 10 s with `max_steps` 0; buffer limit SOURCE

Operational limitation noted in edition 2.0 and kept: `execAsync` buffers the run until exit, with no explicit wall-clock timeout or cancellation path; the schema does not enforce a positive integer for `max_steps`. Use finite validated budgets, streaming capture and a bounded operation lifecycle. [S4]

25 Findings: parser and simavr

ID	Effect	Action and regression	Status
PS09 high	**Boot HEX parser accepts corrupted records.** The custom bootloader parser does not validate checksum, hex digits or record length and continues past EOF records. 2.1 It also fills gaps between regions with `00`, where erased flash is `FF`. `src/ihex_loader.c:6–55, 65, 73–95`	Use a validated parser or implement complete length/type/checksum/EOF handling. Bound sparse address allocation and distinguish erased gaps from intentional zero data. *Regression:* reject bad checksum, short lines, non-hex digits, invalid type length and data after EOF; accept known-good sparse files.	EXECUTED bad checksum, data after EOF, a short line and non-hex digits all returned 0
PS10 medium	**Extended segment addresses use OR instead of addition.** For type-02 segment records, the segment base and record offset can overlap in bit positions. Segment `0x1001` and offset `0x0010` yielded `0x10010` instead of `0x10020`. `src/ihex_loader.c:40, 80`	Compute the specified segment-base-plus-offset address with overflow checks in both passes. Keep type-04 linear-address handling distinct. *Regression:* the fixture of section 29 and crossing/boundary cases.	EXECUTED base `0x10010`
AV01 high	**Pinned GDB stub has address/range defects.** The `m` read branch tests `addr < flashend`, excluding the last flash byte as a starting address. The `M` write branch accepts flash only below `0xFFFF` regardless of larger configured flash. The shown memory branches validate starts, not complete requested spans. `simavr/sim/sim_gdb.c:652–715`	Audit and patch or qualify an updated simavr dependency. Validate full spans, packet lengths, reply buffer capacity and all supported flash-write packet forms. *Regression:* last-byte reads, cross-end reads/writes and `M` writes below/above 64 KiB on stock and virtual profiles; correct rejection without host memory access.	EXECUTED for the read: the last flash byte is unreadable on ATmega328P and ATmega2560; the rest SOURCE

ID	Effect	Action and regression	Status
AV02 high 2.1	**SPM works from anywhere.** simavr carries out SPM wherever the program counter is. On a chip with a boot section, SPM outside it does nothing. `avr_flash.c`	A strict flash mode in Protosim, on by default for loader tests, that reports each of the four as an error with the program counter: SPM outside the boot section, a page write over a page that was not erased, a read of the application section while it is busy, and no wait before the next operation. Record the mode in the evidence bundle. *Regression:* four small firmwares, each making one of the mistakes, fail; the loader of section 17 passes.	EXECUTED a program at address 0 erased and wrote a page
AV03 high 2.1	**A page write needs no erase.** simavr replaces the page with the buffer. A chip can only clear bits: without an erase the result is old AND new.		EXECUTED `F0` then `3C` read back `3C`, not `30`
AV04 high 2.1	**No read-while-write state.** The application section reads back correctly straight after a page write and the busy bit is never set. On a chip it cannot be read, or executed, until it is re-enabled.		EXECUTED
AV05 medium 2.1	**Flash programming takes no time.** Erase and write finish within the instruction. A chip takes 3.7 to 4.5 ms for each, with interrupts off, so every timing and fairness figure from simulation leaves that out.		EXECUTED zero busy polls
AV06 medium 2.1	**The firmware loader drops a chunk that ends on the last flash byte.** It is given the last valid address as its limit; the bytes are dropped with only a log line. A table at the very end of flash, such as a bootloader's service table or Optiboot's version word, does not load. `sim_elf.c:103, 255`	Patch the comparison, or load through Protosim's own reader. *Regression:* an ELF with data in the last four bytes of flash reads back.	EXECUTED “Loading 4 bytes of Flash data at 7ffc would overflow buffer”
AV07 medium 2.1	**The UART is more forgiving than a chip.** Any write to the status register clears the overrun flag; the frame-error flag stays set after the last byte is read; the receiver queues 63 bytes before it reports an overrun, where a chip holds two; and with several bytes queued the receive flag stays set while the interrupt comes one byte time later, so code that polls with interrupts on can take a byte ahead of the handler. `avr_uart.c`	State these in the UART evidence; do not take the absence of overruns in simulation as headroom. A strict option for the queue depth would let F07 test backpressure.	EXECUTED 63 bytes drained after 100 were sent with nobody reading
AV08 medium 2.1	**A reset discards the simulator's own timers.** `avr_reset()` drops every pending cycle timer and restarts the cycle counter at zero. An external reset is not recorded in `MCUSR`. See PS16 and PS17 and the box in section 08.	Re-register after reset; keep a monotonic cycle count outside simavr.	EXECUTED effects; SOURCE
AV09 low 2.1	**Running off the end of flash is a crash.** When the program counter reaches the end of flash the CPU is marked crashed; a chip wraps to address 0. Erased words themselves execute correctly, as the skip instruction they are.	Report it as its own stop reason, not as `cpu_crashed`.	EXECUTED

Other dependency qualification items. Test watchpoints on CPU and peripheral-driven memory changes; verify pause delivery, reconnect, detach and reset behaviour with the exact avr-gdb version. The `DETACHABLE` branch contains an assignment in a condition (`if (avr->state = cpu_Stopped)`, `sim_gdb.c:765`); whether it is compiled and what detach policy is intended need checking before relying on detach. Keep the debugger endpoint local by default. [S5]

These are findings in Protosim's pinned dependency, not a claim that the current upstream simavr has the same behaviour. Upgrade only with regression evidence and record the new pin.

26 Findings: Protoduino scheduler and events

ID	Effect	Action and regression	Status
PD01 high	**Cleanup can remain unscheduled.** `process_destroy` marks EXITING and arms `PT_FINAL` but does not schedule execution. `process_poll` rejects EXITING, and the optional inbox pop also rejects it. With no global event or earlier pending poll, repeated `process_run` calls need not enter cleanup. 2.1 The same holds for a process that ends by itself: `call_process()` arms the finalisation and nothing triggers it. `src/sys/process.c:430–465, 299–347`	Give exiting processes an explicit scheduler service path or reliably queue a cleanup event; define progress when cleanup itself yields. 2.1 One line does it: `process_post(p, PROCESS_EVENT_EXIT, NULL)` after `PT_FINAL()` in `call_process()`. *Regression:* destroy an idle process with empty queues, run the scheduler and require its finally block and unlink to finish; repeat with inbox mode enabled.	EXECUTED a process that ended: after 2.5 s its `PT_FINALLY` had not run, state EXITING
PD02 high	**Unlinking during a sweep skips successors.** `do_poll` and broadcast advance through `pp->next` after dispatch. Finalisation can call `process_unlink()`, which clears that pointer. `process.c:159–185, 273–287, 327–336`	Make traversal robust to removal and lifecycle changes, and preserve/rescan pending work. Saving `next` is a start, but define callbacks that also modify other list entries. *Regression:* finalise the first/middle process during poll and broadcast; verify every intended recipient and the remaining poll flag.	EXECUTED on the AVR for poll: two processes polled, the first ended on its poll, the second was never polled and stayed pending. Broadcast SOURCE
PD03 medium	**Initialisation event failure is ignored.** `process_start` inserts and marks the process before posting INIT, but ignores `process_post`'s result. With a full global queue and inboxes disabled, startup can appear successful without initialisation delivery. 2.1 The default queue has three usable entries, so the fourth process started in a row is affected. `process.c:378–396`	Reserve/guarantee initialisation delivery or roll back and expose failure. Preserve a clear API contract for `process_new` and `process_start`. *Regression:* fill the queue, start a new process and require either exactly one INIT or an explicit rejected start with no active half-initialised entry.	SOURCE
PD04 high	**Queued destination pointers can outlive an instance.** Events hold raw process pointers and dispatch checks only current state. If storage is reused before an old event drains, that event may reach a new instance; module unload can make the pointer invalid. `process.c:325–346, 401–425, 430–443`	Define ownership and lifetime; purge pending events on teardown or add generation-checked handles. Do not release module/storage references while queued work can reach them. *Regression:* queue an event, destroy/recreate in the same storage, and unload a module with pending work; verify no stale delivery. 2.1 The module loader gives each slot one fixed block of RAM, which is exactly this reuse.	SOURCE risk

ID	Effect	Action and regression	Status
PD14 high 2.1	**Polls starve events.** `process_run()` is `if (do_poll()) return; do_event();` : an event is dispatched only when no poll was pending. A process that polls itself on every turn stops all events for good, including the INIT event of every process started after it. And because the serial device polls on every received byte (PD16), events also stop while bytes arrive back to back.	`do_poll(); do_event();` in `process_run()` . *Regression:* with a self-polling process running, a second process receives its INIT; during a serial transfer a process waiting on events is served between bytes.	EXECUTED second process never got INIT; a heartbeat waited 15.1 ms during a transfer, 2.0 ms with the change
PD21 medium 2.1	**A raised error can wait for ever.** `PT_RAISE()` returns `PT_WAITING` and lets the handler run the next time the thread is scheduled. A process is only scheduled by an event or a poll, so a process that raises may never reach its `PT_CATCHANY` block and its error is never reported. A thread with no catch block at all falls off the end of its function, which the compiler reports as “control reaches end of non-void function” for the library's own serial process.	A `PROCESS_RAISE()` that polls the process before raising. *Regression:* a process that raises with empty queues reports its error within one scheduler pass.	EXECUTED a refused module produced no report until the raise was preceded by a poll

27 Findings: Protoduino TUI

ID	Effect	Action and regression	Status
PD05 high	**Same-buffer text edits evade appearance diff.** The signature mixes text address and length, not text bytes. Changing AA to BB in the same two-byte buffer left the signature unchanged. Without explicit dirty marking the changed content can remain invisible. `src/lib/tui/tui_layout.c:797–829`	Document/enforce explicit content invalidation or add a content generation/hash policy. Do not assume equal pointer and length imply equal text. *Regression:* modify content in place without moving/resizing the node and verify the selected API invalidates it; exercise manual dirty marking too.	ISOLATED not re-run; the signature code was re-read
PD06 high	**Removed trailing nodes may leave stale pixels.** The damage pass examines only current elements and then stores the new count. Removing a trailing child while the retained parent keeps the same appearance produced `dirty_any=0` . A fully empty tree also takes an early layout return. `tui_layout.c:833–880` and `tui_compute_layout()`	Track removal/old bounds or invalidate an enclosing retained region on structural changes, including transition to an empty tree. *Regression:* remove the last fixed-position child and then all nodes; verify the old terminal regions are cleared without requiring unrelated changes.	ISOLATED not re-run
PD07 high	**Box glyph writes can silently truncate.** `_put_acs` writes up to three UTF-8 bytes directly to `ipc_pipe_write` and ignores the returned count. The pipe explicitly supports partial writes, so a nearly full buffer can receive only part of a glyph. `tui_render.c:114–120` ; `sys/ipc.c:167–213`	Use a backpressure-aware output path, or reserve capacity and retain a resumable pending write. Ensure renderer progress flags reflect delivered output. *Regression:* zero, one and two bytes of free pipe space before a three-byte border glyph; drain and resume, requiring the exact final UTF-8 sequence.	SOURCE

ID	Effect	Action and regression	Status
PD08 medium	**Pointer hashing contains a 16-bit shift hazard.** The code shifts `uintptr_t p` by 16. On classic AVR, `uintptr_t` and the promoted integer width are 16 bits; shifting by that width is not defined C behaviour. `tui_layout.c:815–823`	Cast to a sufficiently wide unsigned type before shifting, or conditionally omit high-word hashing on 16-bit pointers. *Regression:* compile with the AVR toolchain's shift warnings enabled and run stable-signature fixtures for text and scroll objects.	EXECUTED avr-gcc: “right shift count >= width of type” at lines 818 and 823

28 Findings: Protoduino limits, serial path and build

ID	Effect	Action and regression	Status
PD09 high	**Memory reporting and instance sizes need qualification.** `free_mem` returns signed int after pointer-to-int casts; a gap larger than 32767 bytes cannot be represented by a classic 16-bit int. Separately, `process_type_t.size` is intentionally `uint8_t`, so instances above 255 bytes cannot be represented; the macro has no size assertion. `sys/mem.c:3–7`; `process.h:241–247, 274–288`; `process.c:408–409`	Use an explicit wider result and well-defined address arithmetic for diagnostics. Keep the compact process limit with a compile-time assertion, or version the descriptor ABI if widening it. *Regression:* verify large free gaps in the expanded profile and reject a process struct above the selected limit at compile time.	EXECUTED for the size: a 316-byte struct builds with “large integer implicitly truncated” and is stored as 60; `free_mem` SOURCE
PD10 high	**Loader guide is not an executable design.** The guide packages only lengths and raw `.text` / `.data`, omitting relocations, imports, entry metadata and zero-fill needs. Exact-section extraction misses possible `.text.*` or `.data.*` sections. `avr-ld -r` preserves relocatability; it does not finish linking imports. Native RAM execution contradicts the selected classic AVR architecture. `docs/loader.md`	2.1 Replace the guide with a pointer to the design of section 17, which is flash-resident, defines the supported object (a host-linked `ET_EXEC` without static data) and the ABI, and has running code. avr-gcc reports `-fpic` as unsupported.	SOURCE
PD11 high 2.1	**`CC_EXTERN` defines variables in C.** It is `extern "C"` in C++ and nothing in C, so `CC_EXTERN struct process *process_current;` and `PROCESS_EXTERN()` define the variable in every C file that includes `process.h`. avr-gcc 7 merges the copies; from GCC 10 the link fails. `src/cc.h`	`#define CC_EXTERN extern` in C, and one definition in `process.c`. *Regression:* two C files that include the header link with `-fno-common`.	EXECUTED “multiple definition of `process_current'”
PD12 medium 2.1	**The serial buffers are 16 bytes on every part.** `serial.h` includes the configuration before the header that brings in `avr/io.h`, so `serial.conf.h` sizes the buffers before `RAMEND` is defined, and every part gets the size meant for parts under 1 KB of RAM. `src/sys/serial.h:6–7`	Include `cc.h` first. *Regression:* the ring arrays are 64 bytes on an ATmega328P.	EXECUTED 16 bytes; 64 with `avr/io.h` first
PD13 medium 2.1	**The documented way to set a baud rate has no effect.** `serial.h` says to set `baud` and then call `process_new()`, which clears the struct. The device opens at 9600.	Correct the usage text: set it after `process_new()`. *Regression:* a requested 115200 gives the matching divisor.	EXECUTED `UBRR0L` = 207

ID	Effect	Action and regression	Status
PD15 high 2.1	**The serial process strands output.** On a poll it moves as many bytes from its pipe as the UART ring has room for, then waits. Nothing polls it for the rest. `src/sys/process/serial_private.h`	Poll itself while the pipe holds bytes; this needs PD14 fixed first, or it stops all events. *Regression:* 80 bytes written at once all leave the UART.	EXECUTED 15 of 80 bytes
PD16 low 2.1	**The receive callback has the wrong type.** The serial process installs a function that returns nothing where the serial layer expects one that says whether it consumed the byte, and acts on whatever is left in the result register. The same callback polls the device process on every byte, which feeds PD14.	Return `true`. Poll the consumer, not the device, on receive.	EXECUTED compiler: “incompatible pointer type”; effect SOURCE
PD17 high 2.1	**The clock header rejects every part but three families.** `sys/clock.h` accepts the ATmega328, 168 and 88 and stops with `#error` otherwise, although the AVR clock code has nothing part-specific. Timers do not build for the ATmega2560.	Select on `__AVR__`, as `sys/uart.h` does. *Regression:* a timer example builds for every profile of section 13.	EXECUTED “clock: Unsupported Platform!”
PD18 low 2.1	**The protothread pipe macros do not compile.** `PT_STDIN()` and `PT_STDOUT()` yield the pipe struct, not its address. The `_EX` forms that processes use are fine. `src/sys/pt/pipe.h:11–17`	Take the address in both macros.	EXECUTED
PD19 medium 2.1	**Three defects in the UART and serial layer.** `uartN_baudrate()` reports twice the real rate when double speed is off. `uartN_close()` ends with an unconditional `sei()`. `serialN_flush()` tests the interrupt flag the wrong way round and sends nothing with interrupts off.	Patches in the loader design document, section 7, findings 1 to 3.	EXECUTED 57600: reported 117,647; nine of ten bytes unsent
PD20 medium 2.1	**The UART driver cannot be used without interrupts or cheaply.** The handlers and callback pointers are always compiled in, and `openex()` works out its divisor at run time with 32-bit divisions: 292 bytes for `uart0_open(115200)`. A bootloader has to set the registers itself.	A compile-time divisor macro, an open that only writes registers, a switch for the handlers, and the polled calls as inline functions.	EXECUTED 224 + 68 bytes
PD22 low 2.1	**Smaller items.** The polled receive calls and the receive interrupt split the stream when used together, and nothing says so. A frame-error branch in `uartN_rx_error()` can never run. A baud rate of 0 divides by zero. A `#pragma message` prints on every build. `platformio.ini` names a library by an absolute path on one Windows machine. `lc-addrlabels.h` is an `#error`.	The loader design document, section 7, findings 5 to 9.	EXECUTED the first; the rest SOURCE

Additional limitations to retain in the backlog, from edition 2.0. *Near program-memory reads:* `rune16` uses `pgm_read_word`, and `process_new` reads PROGMEM descriptors with near accessors. These are not automatically broken on today's linked image; larger builds must keep near-read objects in reach or use far-access support. [S7, S10, R8] *Logger ownership:* the rotating `error_pool` deliberately overwrites older entries while queued events carry pointers to its slots. Bursts can replace the contents of unconsumed records. Size tuning alone is not an ownership guarantee; define loss reporting or stable message ownership. [S7] *IPC*

counters: `ipc_pipe_available` and `ipc_pool_count_free` read multi-byte state outside an atomic snapshot. On an 8-bit AVR with concurrent ISR updates, observations may tear. Audit each call context and add atomic snapshots where needed; this was not reproduced on AVR. [S8]

These findings do not imply that every supported application fails. They identify concrete triggers and conditional boundaries that matter when promoting the framework into an AI-operated test and debugging toolbox. The module loader of section 17 runs on the library as it is, with a marked workaround for PD01, PD14 and PD21.

29 Checks executed

29.1 For this edition 2.1

Everything in this table was run. The Protosim and MCP checks are in `documents/checks/` of this repository and reproduce with one command; the Protoduino and simavr checks are in the Protoduino repository beside the loader work.

Check	What ran	Observed
How a run ends PS01, PS11, PS12	Protosim on a fixture, per ending and UART mode, with the real exit status and signal	Step limit on the default bridge: killed by signal 2. The same with `--uart0-out` : 0. UART match: 0. Match never comes: 1. `--help` : “Cannot load ELF --help”.
Out-of-range dump PS05	`--dump-sram 0x7ff00000:16` at a breakpoint	Killed by signal 11
Boot HEX parser PS09, PS10	`protosim_read_ihex()` in a native driver on five fixtures	Bad checksum, data after EOF, short line and non-hex digits: all return 0. Type-02 record: base `0x10010` , not `0x10020` .
Debug session 07, PS07, AV01	avr-gdb 15.0.50, MI mode, against `protosim -d` , ATmega328P and ATmega2560	The table of section 07. A second instance on the port says “listening”. The port accepts a connection on a non-loopback address. The last flash byte cannot be read.
Step budget while stopped PS02	`-d --max-steps 700` , stop at `main` , wait	“MAX-STEPS (700) reached” after 18.3 s without one instruction; the simulator exits under the debugger
MCP server MC01 to MC05, MC07	The server over stdio from a minimal client: the committed bundle and a fresh `tsc` build	Headless run: `breakpoints_hit` empty, `hot_path` null, coverage parsed. Step-limited run: error path, `isError` false. Path with a space split. Existing Mega project overwritten with `board = uno` . `max_steps` 0: no answer in 10 s.
Bootloaders and input PS08, PS13 to PS17, PS19	Protosim with `--bootloader` , HEX and ELF, `--uart0-in` , a watchdog reset, profiling	As listed in section 23. Recorded in `protoduino/docs/elfloader/prototype/results-simulators.txt` .
simavr model AV02 to AV09	Small AVR programs in simavr: SPM from address 0, write without erase, read after write, UART overrun, reset	As listed in section 25. Same result file; sources in `prototype/simprobes/` .
Protoduino scheduler PD01, PD02, PD14, PD21	Sketches on the unmodified library in simavr: a process that ends; two processes polled, the first ending; a self-polling process; a kernel with a heartbeat during a module load, with and without the change to `process_run()`	As listed in section 26. `prototype/probes/run_probes.sh` , `results-library.txt` , `results-modules.txt` .

Check	What ran	Observed
Protoduino serial path and build PD08, PD09, PD11 to PD13, PD15 to PD20	The same probes, and compiles with avr-gcc for ATmega328P and ATmega2560	As listed in sections 27 and 28
Loader 17	13 firmware scenarios and 9 module scenarios in simavr, flash compared with simavr's own ELF reader	All 22 as designed. `results-tests.txt`, `results-modules.txt`.

```
# in this repository: Protosim, the debugger and the MCP server
documents/checks/run_checks.sh          # writes documents/checks/results.txt

# in the Protoduino repository: the library, simavr's model, the loader
cd docs/elfloader/prototype
./make_fixtures.sh && probes/run_probes.sh && ./run_tests.sh && ./run_modules.sh
```

29.2 Isolated checks of edition 2.0

The review of edition 2.0 executed small host programs using the unchanged custom HEX parser or extracted current functions, plus exact JavaScript regex literals from the MCP source. Minimal host structures/stubs isolated the relevant logic. These checks establish the listed local behaviour; they do not establish complete AVR firmware behaviour or overall simulator correctness.

Check	Fixture / expected result	Observed in 2.0	In 2.1
HEX checksum	Data record `:01000000AA00` has an invalid checksum; parser should reject it.	Accepted: return 0, base 0, length 1, first `0xAA`.	The same **EXECUTED**
HEX type-02 address	Segment `0x1001` and offset `0x0010` should address `0x10020`.	Parser reported `0x10010`, matching bitwise OR.	The same **EXECUTED**
Text signature	Same text pointer/length; buffer changes from AA to BB.	Signature equality remained true.	Not re-run
Trailing-node damage	Current fixed root unchanged; previous frame had one extra trailing child.	`dirty_any=0` and `root_dirty=0`.	Not re-run
Poll traversal	Two linked pending processes; dispatch finalises/unlinks the first.	One dispatched; second `needs_poll=1`; `poll_requested=0`.	The same on the simulated AVR with the whole library **EXECUTED**
Breakpoint parser	Feed current BREAKPOINT HIT text to the exact MCP breakpoint regex.	No match.	The same through the running server **EXECUTED**
Hot-path parser	Feed a normal `[CG] Hot Path` header and following path line to the exact regex.	No match.	The same through the running server **EXECUTED**

Portable reproduction recipes. For the segment case, write the records below to a file and call `protosim_read_ihex` directly in a small native C driver that prints return code, base and length. This exercises the custom bootloader parser, not the main firmware HEX parser imported from simavr.

```
:020000021001EB
:01001000AA45
:00000001FF
```

For the TUI checks, retain `_element_sig` and `_compute_damage` with minimal equivalent fields and a stable fixed root. Move these narrow cases into the repository's real native/AVR suites before merging fixes.

No repository source was changed by either review. The reports' examples and findings are intended to guide implementation and regression work.

30 Source index

Repository references are pinned to the reviewed revisions of section 02. Line references in findings refer to these snapshots.

Key	Subject	Files
[S1]	Protosim execution and loading	`src/protosim.c` : main loop, configuration, GDB initialisation, memory copies and stop handling. Also `src/ihex_loader.c`, `src/stk500_uploader.c`, 2.1 `src/uart_pty.c`.
[S2]	Protosim diagnostics	`src/debug.c` : breakpoint logging, SRAM/watch dumps, SREG formatting and symbol lookup.
[S3]	Protosim measurements	`src/profiling.c` : coverage allocation, callgraph and cycle accounting. Also `include/protosim_core.h`.
[S4]	Existing MCP server	`mcp-server/src/index.ts` : tool schemas, subprocess invocation, project generation and output parsing. Also `package.json`, `tsconfig.json`, `scripts/protosim-mcp.sh`, 2.1 `packaging/nfpm.yaml`, `mcp-server/dist/index.js`, `SKILL.md`.
[S5]	Pinned simavr debugger and core	`simavr/sim/sim_gdb.c` : read/write packet handling, watchpoints and listener setup. Also `sim_avr.h/c`, `sim_core.c`, 2.1 `avr_flash.c`, `avr_uart.c`, `sim_elf.c`.
[S6]	Mega2560 device model	`simavr/cores/sim_mega2560.c` and `.h`. AVR-LibC `iom2560.h` confirms RAMSTART `0x0200`, RAMEND `0x21FF`, XRAMEND `0xFFFF` and FLASHEND `0x3FFFF`.
[S7]	Protoduino scheduler and continuations	`src/sys/process.c`, `process.h`, `process.conf.h`, `protoduino-config.h`, `lc.h`, `lc-switch.h`, `lc-addrlabels.h`, 2.1 `pt.h`, `cc.h`.
[S8]	IPC pools and pipes	`src/sys/ipc.c`, `ipc.h`, `ipc.conf.h`, 2.1 `pt/pipe.h`.
[S9]	TUI layout and rendering	`src/lib/tui/tui_layout.c`, `tui_render.c`, `tui.h`, `docs/tui.md`, `docs/tui-roadmap.md`. Findings use implementation, not roadmap claims alone.
[S10]	Memory and Unicode assumptions	`src/sys/mem.c`, `src/lib/text/rune16.c`, `platformio.ini`.
[S11]	Loader proposal	`docs/loader.md` : experimental compilation/package instructions; not an implemented loader.
[S12] 2.1	Serial path and loader work	`src/cpu/avr/uart*.{h,c}`, `src/sys/serial*`, `src/sys/process/serial*`, `src/sys/clock.h` ; and `docs/elf-loader-design.pdf` with `docs/elfloader/overlay/` and `docs/elfloader/prototype/` on the branch named in section 02.

The Protoduino submodule pinned in Protosim differs from the standalone revision used for the Protoduino findings and is not initialised in a fresh checkout. Findings for standalone Protoduino must be checked separately against that pin.

31 Official technical references

- [R1] GDB/MI specification
- [R2] GDB symbols and file/section registration
- [R3] GDB optimised-code limitations
- [R4] ATmega640/1280/1281/2560/2561 datasheet, DS40002211A
- [R5] GCC AVR options and EIND/trampoline conventions
- [R6] Codex MCP configuration, official OpenAI documentation
- [R7] Claude Code MCP configuration
- [R8] AVR-LibC program-space utilities

- [R9] Microchip AVR device debug interfaces
- [R10] 2.1 ATmega48A/88A/168A/328P datasheet, “Self-Programming the Flash”: boot-section permission, erase before write, read-while-write, 3.7 to 4.5 ms per operation. Optiboot 8.3 source for `do_spm`. The Model Context Protocol specification for tool annotations and structured results.

Documentation was accessed in the conversations that produced the two editions. Exact deployment versions and client availability must be qualified during implementation. Requirements and priorities are the report's engineering proposals based on the user's stated direction, not claims made by the referenced projects.

32 Interface examples

Connect the existing locally built MCP server

```
# Replace the absolute path with the local checkout.
claude mcp add --transport stdio protosim -- \
  node /absolute/path/protosim/mcp-server/build/index.js

codex mcp add protosim -- \
  node /absolute/path/protosim/mcp-server/build/index.js
```

These commands register a stdio server; they do not implement the proposed tools. `tsc` writes `build/index.js`; it needs `npm ci` and `npm run build` in `mcp-server/` first. 2.1 Without a build, the committed bundle `mcp-server/dist/index.js` starts with plain `node` and offers the same three tools; an installed package provides it as `protosim-mcp` (MC06). Either way the server looks for `bin/protosim` two directories above itself and then on the PATH. Resolve dependencies in the server environment by capability: Protosim for simulation, build tools for compilation, and avr-gdb for the proposed debugger. Hosted clients need access to that execution environment; a laptop-local path alone is insufficient. [S4, R6, R7]

2.1 With the server as it is, pass `--uart0-out <file>` among the flags of `run_simulation`: it is the only way to get the structured result until PS11 is fixed (MC05).

A debug session as it ran 2.1

```
$ protosim firmware.elf -m atmega328p -d --uart0-out uart.txt &
$ avr-gdb --interpreter=mi2 -q -nx firmware.elf
1-gdb-set mi-async on
1^done
2-target-select remote localhost:1234
*stopped,frame={addr="0x00000000",func="__vectors",args=[],arch="avr:5"},thread-id="1",...
2^connected
3-break-insert main
3^done,bkpt={number="1",type="breakpoint",disp="keep",enabled="y",addr="0x000000e6",
     func="main",file="firmware.c",fullname="...",line="17",...}
4-exec-continue
4^running
*stopped,reason="breakpoint-hit",disp="keep",bkptno="1",frame={addr="0x000000e6",
     func="main",args=[],file="firmware.c",fullname="...",line="17",arch="avr:5"},...
5-data-evaluate-expression counter
5^done,value="4660"
6-break-watch counter
6^done,wpt={number="2",exp="counter"}
7-exec-continue
7^running
*stopped,reason="watchpoint-trigger",wpt={number="2",exp="counter"},
     value={old="4660",new="4764"},frame={addr="0x00000130",func="main",...,line="21",...}
```

Records shortened where marked and wrapped to fit; the checks that produce them are in `documents/checks/`.

Proposed stopped-operation response

```
{
  "schema_version": "1.0",
  "request_id": "req-42",
  "session_id": "session-a",
  "operation_id": "op-7",
  "state": "stopped",
  "state_version": 18,
  "stop_reason": "breakpoint",
  "build_id": "build-07",
  "profile_id": "stock-atmega328p-v1",
  "location": {
    "repository": "protoduino",
    "file": "src/sys/process.c",
    "function": "process_destroy",
    "line": 430
  },
  "child": {"exit_code": null, "signal": null},
  "simulator_model": {"flash": "strict"},
  "test_verdict": "not_evaluated"
}
```

Illustrative identifiers and stop event, not an executed debug session. Implementation should also report ELF/profile hashes, current instruction address with address-space/unit tags, simulated cycles when available, and an evidence reference. 2.1 The `child` and `simulator_model` fields are additions of this edition (09, 21).

Proposed acceptance command

```
protosim --doctor --project /workspace/application \
  --profile stock-atmega328p-v1 --require build,debug,mcp --json
```

Implement this diagnostic command early. It gives both developers and coding agents a shared, inspectable account of the environment before any attempt to explain firmware behaviour.